

Highlights

Bilevel Manifold Fitting

Xuelin Zhang, Hong Chen, Li Shen, Bin Gu, Heng Huang

- Developing a new bilevel scheme for manifold fitting, sample generation, and nonlinear interpolation.
- Establishing the generalization bounds via algorithmic stability for stochastic bilevel minimax algorithm.

Bilevel Manifold Fitting

Xuelin Zhang^{a,b}, Hong Chen^{a,b,*}, Li Shen^c, Bin Gu^d, Heng Huang^e

^a*Engineering Research Center of Intelligent Technology for Agriculture, Ministry of Education, Wuhan, China*

^b*College of Informatics, Huazhong Agricultural University, Wuhan, China*

^c*School of Cyber Science and Technology, Sun Yat-Sen University, Shenzhen, China*

^d*School of Artificial Intelligence, Jilin University, Jilin, China*

^e*Department of Computer Science, University of Maryland College Park, USA*

Abstract

The manifold assumption states that high-dimensional ambient data possess a low-dimensional geometric structure, which has promoted many manifold learning models achieving promising performance for wide applications. However, when **there exists** additive noise or noisy dimensions, it is challenging to estimate the latent manifold since the noisy information tends to mislead the data-driven mapping from the ambient space to the latent space. To address this issue, we formulate a *Bilevel Cycle Generative Adversarial Network (BCGAN)* **that comprises** two generative adversarial networks and a specific manifold fitting module. This network can automatically assign masks to the ambient or latent data, learn robust mutual mappings, and generate new synthetic samples. Theoretically, we establish the upper bounds of generalization error for the stochastic bilevel minimax problems to reveal the relationship between generalization capability and parameter settings. Experiments on both synthetic and real-world datasets verify the competitiveness and robustness of the proposed approach for manifold fitting with corrupted data.

Keywords: Manifold fitting, bilevel optimization, noisy feature, generative adversarial networks, learning theory

*Corresponding author.

Email addresses: xlinml@163.com (Xuelin Zhang), chenh@mail.hzau.edu.cn (Hong Chen), mathshenli@gmail.com (Li Shen), gubin@jlu.edu.cn (Bin Gu), heng@umd.edu (Heng Huang)

1. Introduction

Manifold assumption supposes that high-dimensional data lying in an ambient space $\mathcal{N} \subset \mathbb{R}^{d_0}$ are essentially supported on a latent manifold $\mathcal{M} \subset \mathbb{R}^d$ ($d < d_0$) (Meilă and Zhang, 2023). To fit intrinsic mappings between the ambient space and its low-dimensional manifold, various approaches have been developed to approximate the local and global geometry of a latent manifold with empirical observations lying on or close to it (Fefferman et al., 2018, 2020), where deep neural networks have shown promising fitting performance, e.g., the specific manifold network (Chakraborty et al., 2020), generative adversarial networks (GANs) (Zhen et al., 2021), and variational autoencoders (VAEs) (Miolane and Holmes, 2020). In addition to the strategy of learning a mapping, there is another pathway that considers manifold generation for specific scenarios (Huang et al., 2019; Miolane and Holmes, 2020). Nonetheless, most existing approaches are sensitive to atypical observations, such as additive noises (Yao et al., 2023, 2024b). Their similarity metric involving corrupted samples cannot reflect the data structure objectively and results in a degraded approximation to the latent manifold. Hence, it is crucial to formulate a robust fitting framework that reconstructs the intrinsic geometric structure by removing noisy information from the raw data. Following this line, Yao et al. (2024b) developed a manifold fitting approach based on the cycle-consistent GAN (called MFCGAN), where a specific manifold fitting module is proposed for accurate projection with theoretical guarantees (Yao et al., 2023). Although the manifold fitting module and consistency loss benefit the alignment against additive noise, MFCGAN remains sensitive to noisy dimensions and channels.

Despite impressive progress in manifold fitting, there is scarce work on the impact of noisy features on robustness and generalization. Toward this end, the following fundamental problem remains largely open:

Can we improve the robustness and generalization of manifold fitting under complex noisy circumstances?

In this paper, we answer this question from two perspectives:

- (*How*) Under what strategies do new manifold learning schemes reconstruct the intrinsic mapping between the ambient space and the latent region in the presence of redundant and noisy features?
- (*Why*) What theoretical guarantees does the proposed approach exhibit besides empirical performance gains?

Intuitively, we can handle the above questions using a two-stage framework, i.e., selecting informative features **first** (e.g., wavelet transformation (Antoine et al., 2010; Deutsch et al., 2016)), and then implementing a robust manifold fitting approach (e.g., robust GANs (Zhen et al., 2021; Yao et al., 2024b)). However, under such a strategy, the empirical set involving informative dimensions is fixed without considering the downstream tasks. Naturally, it is necessary to design an iterative algorithm for automatically updating the selected set of informative dimensions in terms of the model’s predictive accuracy. Inspired by the masked autoencoder for image recovery (He et al., 2022) and meta learning for coreset selection (Borsos et al., 2020; Zhou et al., 2022; Hao et al., 2024), this paper considers assigning masks on the ambient space, where truly informative features are left for modeling and reconstructing the manifolds.

However, several challenges **lie along** this route.

- It is NP-hard to learn the discrete mask variables directly, **which take** values in $\{0, 1\}$, especially for high-dimensional data.
- Most manifold fitting and bilevel optimization algorithms require a complex optimization strategy and **numerous** computation steps (Huang et al., 2019; Ji et al., 2021), which often lead to a **significant** computational burden.
- It is **challenging** to design a feasible optimization algorithm for solving bilevel min-max problems, where the hypergradient w.r.t. **the** lower-level min-max problem is **complex** to approximate.

1.1. Contributions

To address the aforementioned challenges, a bilevel probabilistic optimization strategy equipped with projected gradient estimation (PGE) (Zhou et al., 2022) is designed to simplify hypergradient computation and assign dimensional masks efficiently. We integrate the meta-learning scheme and cycle consistency metric into the manifold-based generative adversarial framework to formulate a new *Bilevel Cycle Generative Adversarial Network (BCGAN)*, **which enables simultaneous realization of** automatic dimensional masking, effective synthetic data generation, and robust manifold estimation.

The core strategy is to iteratively update the informative feature set as well as the mappings between the ambient and latent spaces, where **a probabilistic bilevel framework automatically learns the masks of BCGAN**. In addition, the cycle consistency metric (Zhu et al., 2017) as well as the specific

Table 1: Properties of our BCGAN and related models ($\checkmark$: enjoying the given information, and $\times$: not available for the information). DE, IT, MF and CS stand for Density Estimation, Image Translation, Manifold Fitting and Coreset Selection, respectively.

	MADE	DAE	CycleGAN	MFCGAN	PBCS	BCGAN (ours)
Learning Task	DE	DE	IT	MF	CS	MF
Optimization Framework	Single-level	Single-level	Single-level	Single-level	Bilevel	Bilevel
New Sample Generation	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	$\checkmark$
Computing Acceleration	$\checkmark$	$\times$	$\times$	$\checkmark$	$\checkmark$	$\checkmark$
Additive Noise Robustness	$\checkmark$	$\times$	$\checkmark$	$\checkmark$	$\times$	$\checkmark$
Noisy Dimension Robustness	$\times$	$\times$	$\times$	$\times$	$\checkmark$	$\checkmark$
Generalization Analysis	$\times$	$\times$	$\times$	$\times$	$\times$	$\checkmark$

manifold fitting module (Yao et al., 2023) are introduced to achieve efficient alignment between the latent manifold $\mathcal{M}$ and the estimated manifold $\widehat{\mathcal{M}}$ (or the ambient space $\mathcal{N}$ and the estimated space $\widehat{\mathcal{N}}$).

The main contributions of this paper are summarized below:

- *Robust bilevel manifold modeling.* The proposed BCGAN employs a dimensional masking mechanism and deep generative models to achieve robust manifold fitting and synthetic sample generation, thereby mitigating the adverse effects of noisy data (e.g., label noise and noisy dimensions).
- *Theoretical guarantees on optimization and generalization.* The designed BCGAN exploits the cycle consistency metric and an efficient policy gradient estimation with the projection operation to reduce computational complexity. Notably, the former accelerates the affiliation of the original and empirical estimated regions, while the latter avoids the heavy computation on Hessian or Jacobian matrices. Theoretically, we establish the generalization bounds for the proposed BCGAN.
- *Empirical competitiveness on manifold learning.* Experiments on synthetic and real-world benchmarks verify that our BCGAN achieves competitive performance in identifying the truly informative features, robust manifold estimation, manifold denoising, and non-linear interpolation.

1.2. Review of the Related Works

Traditional Manifold Learning Approaches. Early research in manifold learning was rooted in numerical analysis and differential geometry, focusing on extracting intrinsic low-dimensional structures from high-dimensional

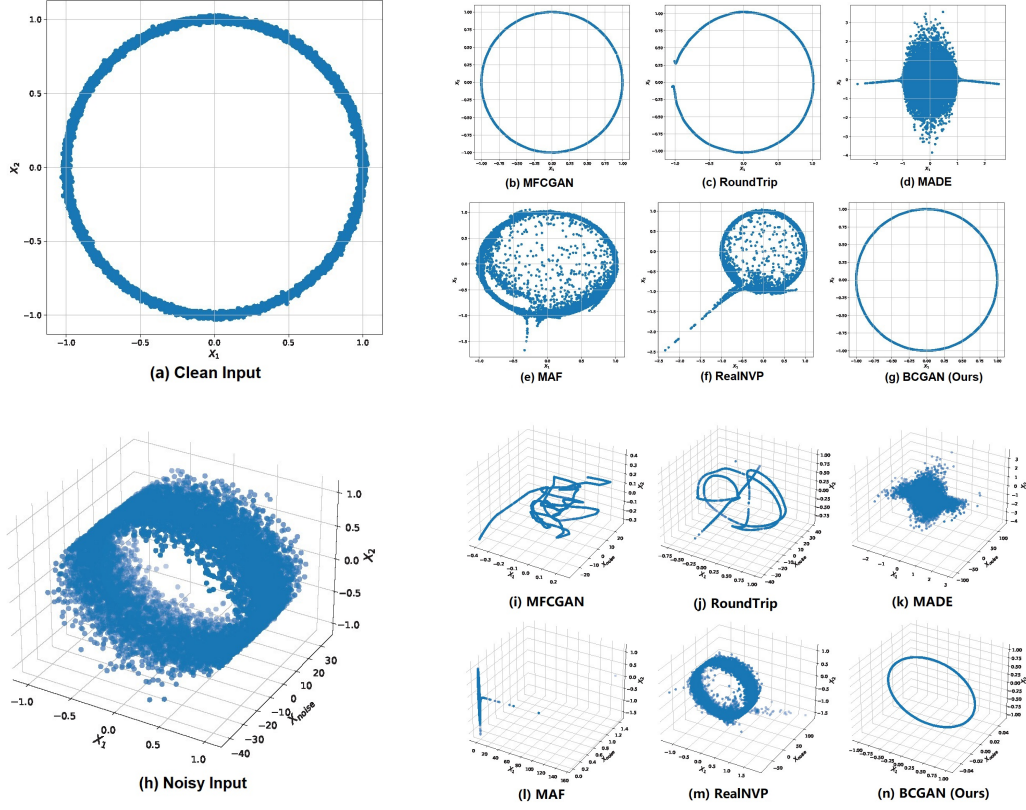


Figure 1: Toy examples on highlighting the impact of noisy features. Clean circle dataset includes two inputs $[X_1, X_2]$, while the corrupted circle dataset includes $[X_1, X_2, X_{noise}]$, where additional noisy input $X_{noise} \in \mathcal{N}(0, 100)$. Empirical comparisons with several baselines (MFCGAN(Yao et al., 2024b), RoundTrip(Liu et al., 2021), MADE(Germain et al., 2015), MAF(Papamakarios et al., 2017), RealNVP(Dinh et al., 2017)) on clean or noisy scenarios are presented in the upper and lower panels, respectively. **The comparisons demonstrate that the bilevel masking mechanism of BCGAN is effective in handling noisy features (X_{noise}).** See Section 5 for details.

98 data, e.g., Principal Curves and Surfaces (Hastie and Stuetzle, 1989) and
 99 Kernel PCA (Schölkopf et al., 1998). Subsequently, ISOMAP (Tenenbaum
 100 et al., 2000) and diffusion maps (Coifman and Lafon, 2006), were introduced
 101 to preserve non-Euclidean structures via geodesic distances or Markovian
 102 connectivity. To enhance robustness, further studies explored noise removal
 103 through wavelet transformations (Antoine et al., 2010), graph-based diffusion
 104 processes (Hein and Maier, 2006), and local approximation methods (Harris

et al., 2011; Wang and Carreira-Perpinán, 2010). Papaioannou et al. (2022) proposed a three-tier framework that combines manifold learning for embedding and geometric harmonics for lifting forecasts back to the original space, which offers computational feasibility and interpretability. Despite their theoretical elegance, these traditional approaches often treat all dimensions in the ambient space as inputs for the kernel-based embedding and lifting processes, which can be sensitive to corruptions such as additive noise, pixel noise, and noisy dimensions or channels (Sober and Levin, 2020; Luo and Hu, 2020).

Manifold fitting via neural networks. Manifold fitting focuses on learning the mappings between the two regions to reconstruct a smooth manifold, which **attempts** to approximate the local and global geometry of a latent manifold using existing observations that **lie on** or close to it (Feferman et al., 2018, 2020). Specifically, the target is to uncover a smooth mapping $F : \mathcal{N} \subset \mathbb{R}^{d_0} \rightarrow \widehat{\mathcal{M}} \subset \mathbb{R}^d$ as well as the corresponding inverse $F^{-1} : F(\mathcal{N}) \subset \widehat{\mathcal{M}} \rightarrow \mathcal{N}$. Recently, there has been an increasing number of studies (e.g., VAEs and GANs) dedicated to reconstructing manifolds, which show tremendous promise in density estimation (Vincent et al., 2008; Germain et al., 2015), image translation (Papamakarios et al., 2017; Zhu et al., 2017) and manifold fitting (Yao et al., 2024b). Under the assumption that partial destruction to inputs brings similar encoding and decoding representations, a denoising autoencoder (DAE) (Vincent et al., 2008) is developed for robust manifold fitting against potentially noisy channels (dimensions). Another line of concurrent work adopts the principle of masked autoencoder to design algorithms (e.g., the masked autoencoder for distribution estimation (MADE) (Germain et al., 2015) and masked autoregressive flow (MAF) (Papamakarios et al., 2017)) for density estimation, which can be applied to the manifold fitting tasks with slight modifications.

To ensure the property of nearly mutual mappings for image-to-image translation, Zhu et al. (2017) developed the popular CycleGAN with cycle-consistent loss to alleviate the previous dependence on paired images. Yao et al. (2024b) further leverage the modified CycleGAN (MFCGAN) to approximate the near-bijective mappings between the ambient and latent regions by training the two generators as the exponential map and logarithmic map supported on the latent manifold. However, the implementations of DAE, MADE and MAF heavily depend on the ambient dimension D for point projection, which brings high computation costs. Moreover, both Cy-

cleGAN and MFCGAN are sensitive to noisy dimensions in raw data. Unlike the above approaches, our formulation concerns the computational efficiency and robustness for manifold learning with noisy dimensions.

Bilevel scheme for coreset selection. The meta-based masking policy was introduced by Borsos et al. (2020), where a bilevel neural network was designed for coreset selection. An enhanced version incorporating a probabilistic optimization strategy was later developed for classification tasks involving imperfect training data (Zhou et al., 2022; Hao et al., 2024), particularly for scenarios with corrupted images, imbalanced categories, and label noise. Indeed, the probabilistic bilevel coreset selection (PBCS) in (Zhou et al., 2022) has not been applied to the manifold-valued scenarios with noisy features. To the best of our knowledge, there has been no prior endeavor to explore the bilevel masking policy for robust manifold fitting.

To better highlight the novelty of our BCGAN, we summarize its properties in Table 1 compared with several related state-of-the-art models, including MADE (Germain et al., 2015), DAE (Vincent et al., 2008), CycleGAN (Zhu et al., 2017), MFCGAN (Yao et al., 2024b) and PBCS for feature selection (Zhou et al., 2022). Moreover, a toy example of manifold fitting is provided in Figure 1 to show the fitting performance with or without additional noisy dimensions. Table 1 and Figure 1 demonstrate that the proposed BCGAN exhibits desirable properties, including robustness against additive label noise and noisy dimensions, computational efficiency and theoretical guarantees.

2. Revisiting the MFCGAN Model

2.1. Problem Setup

Our first target is to approximate the mappings between the ambient space and latent space, while keeping their nearly bijective properties via the MFCGAN (Zhu et al., 2017; Yao et al., 2024b). Before introducing the basic MFCGAN, we first present some necessary assumptions and settings throughout this paper, which have been adopted widely in manifold learning for model construction (Zhen et al., 2021; Yao et al., 2024b) and theoretical analysis (Mohammed and Narayanan, 2017; Fefferman et al., 2018; Sober and Levin, 2020; Fefferman et al., 2020; Yao et al., 2023).

Basic Assumptions. Assume that there are the ambient domain $\mathcal{X} \subseteq \mathbb{R}^{d_0}$ and the latent domain $\mathcal{Y} \subseteq \mathbb{R}^d, d < d_0$. Under the additive noise assumption, the observed random variable $X \in \mathcal{X}$ can be divided into an unobserved

variable $\mathcal{V}$ and a noise component ϵ . Denote ω as the distribution of $\mathcal{V}$ supported on the latent manifold $\mathcal{M}$, where $\mathcal{M}$ is a compact d -dimensional sub-manifold and twice-differentiable. As stated in (Zhu et al., 2017; Yao et al., 2024b), suppose that ω satisfies the uniform distribution on $\mathcal{M}$ w.r.t. d -dimensional Hausdorff measure and noise ϵ lies in the ambient space, where ϵ is independent of $\mathcal{V}$ and follows the distribution

$$\phi_\sigma(\epsilon) = \left(\frac{1}{2\pi\sigma^2} \right)^{\frac{d_0}{2}} \exp \left(-\frac{\|\epsilon\|_2^2}{2\sigma^2} \right).$$

Consider the input set $\{x_i\}_{i=1}^n$ in the ambient space, where $x_i = v_i + \epsilon_i$ for $i = 1, \dots, n$. The sample pairs $\{(x_i, v_i, \epsilon_i)\}_{i=1}^n$ are independent and identically distributed (i.i.d.) realizations of variable pair $(X, \mathcal{V}, \epsilon)$. Naturally, the distribution of X can be regarded as the convolution $\rho = \omega \star \phi_\sigma$ (Yao et al., 2023, 2024b), whose density at point x is

$$\rho(x) = \int_{\mathcal{M}} \phi_\sigma(x - v) \omega(z) dv.$$

To better fit the nearly mutual mappings between $\mathcal{X}$ and $\mathcal{Y}$, Yao et al. (2024b) formulate a novel MFCGAN framework by integrating the fundamental CycleGAN model (Zhu et al., 2017) with a manifold fitting module.

2.2. MFCGAN

Model Structure. As shown in Fig.2, MFCGAN for manifold fitting (Yao et al., 2024b) mainly involves two generators denoted by $G_{\mathcal{X}} : \mathcal{X} \rightarrow \mathcal{Y}$ and $G_{\mathcal{Y}} : \mathcal{Y} \rightarrow \mathcal{X}$, two discriminators written as $D_{\mathcal{X}}$ and $D_{\mathcal{Y}}$, and a specific manifold fitting module $F_{\mathcal{M}}$ (Yao et al., 2023). Before the training process, we collect input x and latent representation y drawn from two domains $\mathcal{X}$ and $\mathcal{Y}$, respectively. Synthetic output $\tilde{y}$ in the domain $\mathcal{Y}$ can be directly obtained through $\tilde{y} = G_{\mathcal{X}}(x)$. In contrast, synthetic input $\tilde{x}$ lying in the domain $\mathcal{X}$ is generated with additive random noise, i.e., $\tilde{x} = G_{\mathcal{Y}}(y) + \epsilon$. Two discriminators, $D_{\mathcal{X}}$ and $D_{\mathcal{Y}}$, are introduced to distinguish $\tilde{x}$ from x and $\tilde{y}$ from y , respectively. Before transforming data to $D_{\mathcal{X}}$, a specific module $F_{\mathcal{M}}$ is introduced to realize pseudo manifold fitting (Yao et al., 2023, 2024a,b). This paper also employs the submodule $F_{\mathcal{M}}$ to accelerate the training process and improve the alignment of realistic samples with the latent manifold $\mathcal{M}$.

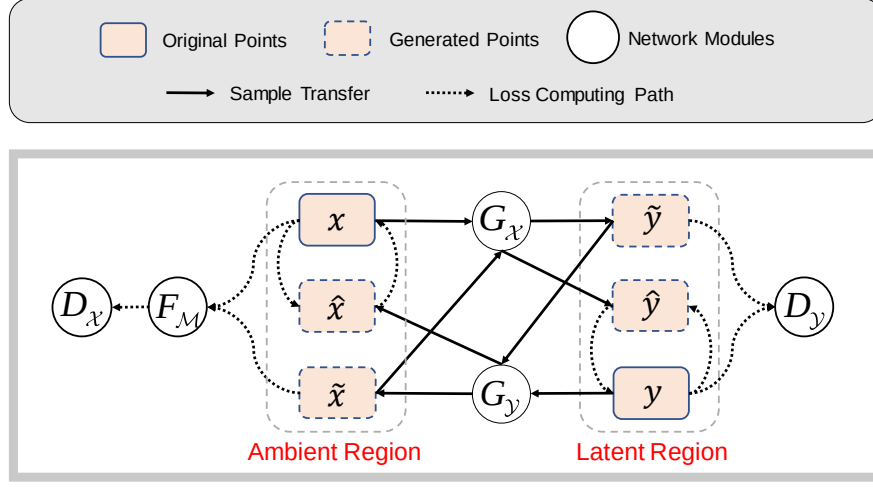


Figure 2: The architecture and computing process of MFCGAN.

206 *Objective Functions.* The objective function mainly involves two adver-
 207 sarial losses $\mathcal{L}_{\mathcal{X} \rightarrow \mathcal{Y}}(G_{\mathcal{X}}, D_{\mathcal{Y}})$ and $\mathcal{L}_{\mathcal{Y} \rightarrow \mathcal{X}}(G_{\mathcal{Y}}, D_{\mathcal{X}}, F_{\mathcal{M}})$, and the cycle consi-
 208 stency loss $\mathcal{L}_c(G_{\mathcal{X}}, G_{\mathcal{Y}})$. The least-square GAN loss (Zhu et al., 2017) is ex-
 209 ploited to stabilize the training process and generate credible results. Given
 210 data $\{x_i, y_i\}_{i=1}^n$, the loss function for evaluating the generated outputs $\{y, \tilde{y}\}$
 211 w.r.t. modules $G_{\mathcal{X}}, D_{\mathcal{Y}}$, is defined by

$$\mathcal{L}_{\mathcal{X} \rightarrow \mathcal{Y}}(G_{\mathcal{X}}, D_{\mathcal{Y}}) = \frac{1}{n} \sum_{i=1}^n [D_{\mathcal{Y}}(\tilde{y}_i)]^2 + [D_{\mathcal{Y}}(y_i) - 1]^2,$$

212 and the loss function w.r.t. $G_{\mathcal{Y}}, D_{\mathcal{X}}, F_{\mathcal{M}}$ is

$$\mathcal{L}_{\mathcal{Y} \rightarrow \mathcal{X}}(G_{\mathcal{Y}}, D_{\mathcal{X}}, F_{\mathcal{M}}) = \frac{1}{n} \sum_{i=1}^n [D_{\mathcal{X}} \circ F_{\mathcal{M}}(\tilde{x}_i)]^2 + [D_{\mathcal{X}} \circ F_{\mathcal{M}}(x_i) - 1]^2.$$

213 Denote the generated samples $\hat{x}_i = G_{\mathcal{Y}} \circ G_{\mathcal{X}}(x_i)$ and $\hat{y}_i = G_{\mathcal{X}} \circ G_{\mathcal{Y}}(y_i)$.
 214 To limit the searching range of mappings, we further consider the cycle con-
 215 sistency loss

$$\mathcal{L}_c(G_{\mathcal{X}}, G_{\mathcal{Y}}) = \frac{1}{n} \sum_{i=1}^n \|x_i - \hat{x}_i\|_1 + \|y_i - \hat{y}_i\|_1, \quad (1)$$

216 which helps to improve the algorithmic robustness against additive noise ϵ
 217 (Zhu et al., 2017).

218 Given the dataset $\mathcal{D} = \{x_i, y_i\}_{i=1}^n$, the whole objective function of MFC-
 219 GAN (Yao et al., 2024b) is

$$\begin{aligned} & \mathcal{L}^{\mathcal{D}}(G_{\mathcal{X}}, G_{\mathcal{Y}}, F_{\mathcal{M}}, D_{\mathcal{X}}, D_{\mathcal{Y}}) \\ &= \mathcal{L}_{\mathcal{X} \rightarrow \mathcal{Y}}(G_{\mathcal{X}}, D_{\mathcal{Y}}) + \mathcal{L}_{\mathcal{Y} \rightarrow \mathcal{X}}(G_{\mathcal{Y}}, D_{\mathcal{X}}, F_{\mathcal{M}}) + \lambda \mathcal{L}_c(G_{\mathcal{X}}, G_{\mathcal{Y}}) \end{aligned} \quad (2)$$

220 where $\lambda < 0$ is a negative parameter that strikes a tradeoff between the ad-
 221 versarial and cycle consistency losses. The optimization scheme of MFCGAN
 222 in (Yao et al., 2024b) can be formulated as

$$G_{\mathcal{X}}^*, G_{\mathcal{Y}}^*, F_{\mathcal{M}}^*, D_{\mathcal{X}}^*, D_{\mathcal{Y}}^* = \arg \max_{G_{\mathcal{X}}, G_{\mathcal{Y}}} \arg \min_{F_{\mathcal{M}}, D_{\mathcal{X}}, D_{\mathcal{Y}}} \mathcal{L}^{\mathcal{D}}(G_{\mathcal{X}}, G_{\mathcal{Y}}, F_{\mathcal{M}}, D_{\mathcal{X}}, D_{\mathcal{Y}}). \quad (3)$$

223 Furthermore, Eq.(3) can be parameterized by

$$\boldsymbol{\theta}^* = (\boldsymbol{\theta}_2^*, \boldsymbol{\theta}_1^*) \in \arg \max_{\boldsymbol{\theta}_2} \arg \min_{\boldsymbol{\theta}_1} \mathcal{L}^{\mathcal{D}}(\boldsymbol{\theta}), \quad (4)$$

224 where $\boldsymbol{\theta} = (\boldsymbol{\theta}_2, \boldsymbol{\theta}_1)$ stand for the parameters of modules $\{G_{\mathcal{X}}, G_{\mathcal{Y}}\}$ and
 225 $\{F_{\mathcal{M}}, D_{\mathcal{X}}, D_{\mathcal{Y}}\}$, respectively.

226 Despite achieving the promising performance, the MFCGAN (Yao et al.,
 227 2024b) is vulnerable when facing corrupted data with noisy dimensions, e.g.,
 228 noisy inputs or outputs (see Figure 1). To address this issue, we **attempt to**
 229 **mitigate** the negative impacts of noisy dimensions by assigning a mask value
 230 of 0 to the irrelevant or redundant features (Zhou et al., 2022).

231 3. Bilevel CycleGAN for Manifold Fitting

232 3.1. Probabilistic Bilevel Framework

233 To improve the robustness of CycleGAN-based manifold fitting, we in-
 234 troduce a bilevel optimization framework with a meta learner at the upper
 235 level, which learns the masks on the ambient variable $X \in \mathcal{X}$ (or the latent
 236 variable $Y \in \mathcal{Y}$) to remove the impact of noisy dimensions. Let the mask
 237 $\mathbf{m} \in \{0, 1\}^p$ be a binary vector learned by the upper problem, where $p = d_0$
 238 when assigning $\mathbf{m}$ on $\mathcal{X}$ (or $p = d$ for $\mathcal{Y}$). $\mathbf{m}_i = 1$ implies the i -th fea-
 239 ture is selected as the informative one and otherwise ignored. Let $\boldsymbol{\theta}$ be the
 240 parameter of **the** decision function in the lower level depending on $\mathbf{m}$, and

$$\mathcal{C}_m = \{\mathbf{m} : \mathbf{m}_i \in \{0, 1\}, \|\mathbf{m}\|_0 \leq C, i = 1, 2, \dots, p\}$$

241 is the feasible region of $\mathbf{m}$ with a given positive constant C . Here, the mask
 242 $\mathbf{m}$ is assigned on the inputs $(x_i, \hat{x}_i, \tilde{x}_i)$ when solving the lower-level problem,
 243 where $\tilde{y}_i = G_{\mathcal{X}}(\mathbf{m} \odot x_i)$, $\hat{x}_i = G_{\mathcal{Y}} \circ G_{\mathcal{X}}(\mathbf{m} \odot x_i)$.

244 The discrete bilevel CycleGAN framework is formulated as follows.

245 *Upper Level Problem.* Given the meta dataset $\mathcal{D}_{meta} = \{x_i, y_i\}_{i=1}^{m_1}$, we
 246 formulate the following discrete optimization problem

$$\mathbf{m} = \arg \min_{\mathbf{m} \in \mathcal{C}_m} \mathcal{L}^{\mathcal{D}_{meta}}(\boldsymbol{\theta}^*(\mathbf{m})), \quad (5)$$

247 where

$$\begin{aligned} \mathcal{L}^{\mathcal{D}_{meta}}(\boldsymbol{\theta}^*(\mathbf{m})) = & \frac{1}{m_1} \sum_{i=1}^{m_1} ([D_{\mathcal{Y}}(\tilde{y}_i)]^2 + [D_{\mathcal{Y}}(y_i) - 1]^2 + [D_{\mathcal{X}} \circ F_{\mathcal{M}}(\tilde{x}_i)]^2 \\ & + [D_{\mathcal{X}} \circ F_{\mathcal{M}}(x_i) - 1]^2 + \lambda \|x_i - \hat{x}_i\|_1 + \lambda \|y_i - \hat{y}_i\|_1). \end{aligned}$$

248 *Lower Level Problem.* Given the training set $\mathcal{D}_{train} = \{x_i, y_i\}_{i=m_1+1}^{m_1+m_2}$, the
 249 lower level problem follows Eq.(4) is

$$\boldsymbol{\theta}^*(\mathbf{m}) \in \arg \max_{\boldsymbol{\theta}_2} \arg \min_{\boldsymbol{\theta}_1} \mathcal{L}^{\mathcal{D}_{train}}(\boldsymbol{\theta}; \mathbf{m}), \quad (6)$$

250 where

$$\begin{aligned} \mathcal{L}^{\mathcal{D}_{train}}(\boldsymbol{\theta}; \mathbf{m}) = & \frac{1}{m_2} \sum_{i=m_1+1}^{m_1+m_2} ([D_{\mathcal{Y}}(\tilde{y}_i)]^2 + [D_{\mathcal{Y}}(y_i) - 1]^2 + [D_{\mathcal{X}} \circ F_{\mathcal{M}}(\mathbf{m} \odot \tilde{x}_i)]^2 \\ & + [D_{\mathcal{X}} \circ F_{\mathcal{M}}(\mathbf{m} \odot x_i) - 1]^2 + \lambda \|\mathbf{m} \odot (x_i - \hat{x}_i)\|_1 + \lambda \|y_i - \hat{y}_i\|_1). \end{aligned}$$

251 In general, it is intractable to solve the above discrete bilevel problem
 252 directly. Fortunately, we can formulate its continuous form with the help of
 253 policy gradient estimation (Zhou et al., 2022), where the discrete parameter
 254 space is transformed into the continuous probability space (Zhao et al., 2023).
 255 We consider $\mathbf{m}_i$ as a Bernoulli random variable $\mathbf{m}_i \sim \text{Bern}(\mathbf{s}_i)$, where $\mathbf{s}_i \in$
 256 $[0, 1]$ represents the probability of $\mathbf{m}_i = 1$. Denote the new domain on
 257 probability variable $\mathbf{s} \in \mathbb{R}^p$ as

$$\mathcal{C}_s = \{\mathbf{s} : 0 \leq \mathbf{s}_i \leq 1, \|\mathbf{s}\|_1 \leq C, i = 1, 2, \dots, p\}.$$

258 The above discrete bilevel optimization problem can be relaxed into the
 259 following probabilistic expected form

$$\begin{aligned} \min_{\mathbf{s} \in \mathcal{C}_s} \Phi(\mathbf{s}, \boldsymbol{\theta}) &:= \mathbb{E}_{p(\mathbf{m}|\mathbf{s})} \mathcal{L}^{\mathcal{D}_{meta}}(\boldsymbol{\theta}^*(\mathbf{m})), \\ \text{s.t. } \boldsymbol{\theta}^*(\mathbf{m}) &\in \arg \max_{\boldsymbol{\theta}_2} \arg \min_{\boldsymbol{\theta}_1} \mathcal{L}^{\mathcal{D}_{train}}(\boldsymbol{\theta}; \mathbf{m}), \end{aligned} \quad (7)$$

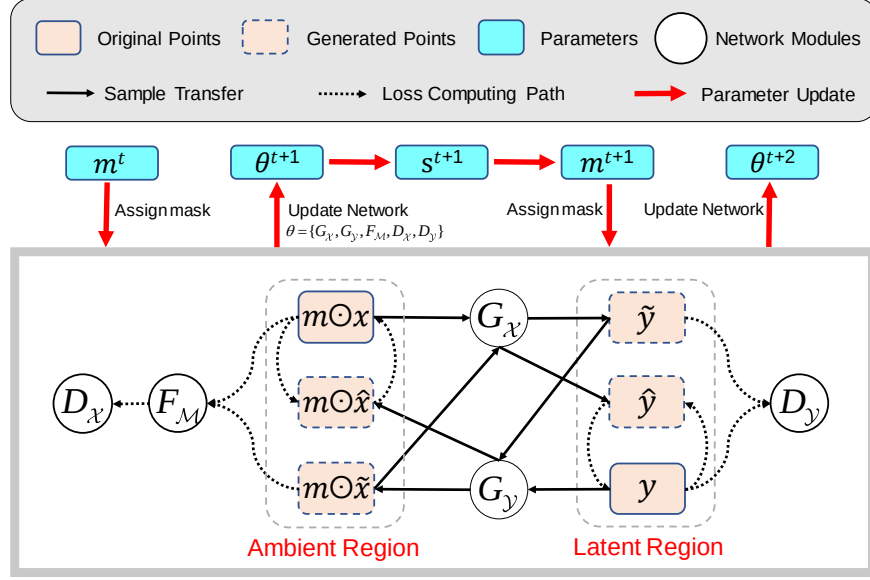


Figure 3: The architecture and computing process of BCGAN. Mask $\mathbf{m}$ could also be assigned to outputs.

where loss function $\ell(\cdot)$ forms the object function in Eq.(5) and Eq.(6), i.e.,

$$\mathcal{L}^{\mathcal{D}_{meta}}(\theta^*(\mathbf{m})) = \frac{1}{m_1} \sum_{i=1}^{m_1} \ell(\mathbf{s}, \theta_1, \theta_2; x_i, y_i).$$

The iterative optimization process of BCGAN is illustrated in Figure 3. Unlike DAE (Vincent et al., 2008) removing a fixed percentage of features, BCGAN can effectively eliminate noise from the original space. We obtain two estimated generators $\hat{G}_x$ and $\hat{G}_y$ w.r.t. the expected G_x^*, G_y^* , and the masks $\mathbf{m}$ in the input region $\mathcal{X}$ or output $\mathcal{Y}$ after training. By controlling the distribution on the latent space $\mathcal{Y}$, the proposed BCGAN can be applied to the following tasks:

- *Manifold Reconstruction.* The desired maps $\hat{G}_y$ and logarithmic map $\hat{G}_x$ are used to empirically reconstruct $\mathcal{M}$ by $\hat{\mathcal{M}} = \hat{G}_y \circ \hat{G}_x(\cdot)$.
- *Noise Reduction.* By applying $\hat{G}_y \circ \hat{G}_x$ to any observed x_i , we obtain a denoised representation $\hat{x}_i$ that lies on the reconstructed manifold $\hat{\mathcal{M}}$.
- *Sample Generation and Nonlinear Interpolation.* To explore the intermediate states between inputs x_i and x_j , their corresponding latent

Algorithm 1 SSGDA for solving BCGAN

Input: Meta set $\mathcal{D}_{meta} = \{x_i, y_i\}_{i=1}^{m_1}$ and training set $\mathcal{D}_{train} = \{x_i, y_i\}_{i=m_1+1}^{m_1+m_2}$, the total number of iterations T , step sizes η , γ_1 and γ_2 .

Initialization: $\mathbf{s}^0$, $\boldsymbol{\theta}_1^0$ and $\boldsymbol{\theta}_2^0$.

- 1: **for** $t = 0$ to $T - 1$ **do**
- 2: Uniformly sample $\mathbf{z}_i = (x_i, y_i) \in \mathcal{D}_{meta}$ and $\mathbf{z}_j = (x_j, y_j) \in \mathcal{D}_{train}$
- 3: $\mathbf{s}^{t+1} = \mathcal{P}_{\mathcal{C}_s}(\mathbf{s}^t - \eta \nabla_{\mathbf{s}} \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t; \mathbf{z}_i))$
- 4: $\boldsymbol{\theta}_1^{t+1} = \boldsymbol{\theta}_1^t - \gamma_1 \nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t; \mathbf{z}_j)$
- 5: $\boldsymbol{\theta}_2^{t+1} = \boldsymbol{\theta}_2^t + \gamma_2 \nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^{t+1}, \boldsymbol{\theta}_2^t; \mathbf{z}_j)$
- 6: **end for**

Output: $\hat{\mathbf{s}} = \mathbf{s}^T$, $\hat{\boldsymbol{\theta}}_1 = \boldsymbol{\theta}_1^T$ and $\hat{\boldsymbol{\theta}}_2 = \boldsymbol{\theta}_2^T$.

representations $\hat{y}_i = \hat{G}_{\mathcal{X}}(x_i)$ and $\hat{y}_j = \hat{G}_{\mathcal{X}}(x_j)$ can be further fed into $\hat{G}_{\mathcal{Y}}$ to realize nonlinear interpolation.

Remark 1. Indeed, $\Phi(\mathbf{s}, \boldsymbol{\theta})$ in (7) should be rewritten by $\Phi(\mathbf{s}, \boldsymbol{\theta}(\mathbf{s}))$, where the learned network parameter $\boldsymbol{\theta}$ is dependent on $\mathbf{m}$ and $\mathbf{s}$. The reason for such simplification can be illustrated by (9) with the efficient PGE optimization and probabilistic representation, where the gradient $\nabla_{\mathbf{s}} \Phi(\mathbf{s}, \boldsymbol{\theta}(\mathbf{s}))$ does not involve $\partial \boldsymbol{\theta}(\mathbf{s}) / \partial \mathbf{s}$.

Remark 2. Under the independent assumption on variable $\mathbf{m}_i$, we can derive its distribution as $p(\mathbf{m} | \mathbf{s}) = \prod_{i=1}^p (\mathbf{s}_i)^{\mathbf{m}_i} (1 - \mathbf{s}_i)^{(1-\mathbf{m}_i)}$. Since $\mathbb{E}_{\mathbf{m} \sim p(\mathbf{m}|\mathbf{s})} \|\mathbf{m}\|_0 = \sum_{i=1}^p \mathbf{s}_i$, the original constraint on $\mathcal{C}_m$ is further transformed into $\mathcal{C}_s$.

3.2. Optimization Algorithm of BCGAN

Now we present the optimization algorithm for assigning masks on the ambient domain $\mathcal{X} \subseteq \mathbb{R}^{d_0}$. Initialize the parameters of $(G_{\mathcal{X}}, G_{\mathcal{Y}}; F_{\mathcal{M}}, D_{\mathcal{X}}, D_{\mathcal{Y}})$ as $\boldsymbol{\theta}^0 := \{\boldsymbol{\theta}_2^0, \boldsymbol{\theta}_1^0\}$, the mask variable as $\mathbf{m}^0 \in \mathbb{R}^{d_0}$, and the corresponding probability variable $\mathbf{s}^0 \in \mathbb{R}^{d_0}$. Algorithm 1 states the computing steps of probabilistic BCGAN through single time-scale stochastic gradient descent ascent (SSGDA). The optimization procedure of (7) at the t -th iteration mainly involve:

Step 1: Computing network parameter $\boldsymbol{\theta}^t$ with given $\mathbf{m}^{t-1}$.

$$\boldsymbol{\theta}^t \in \arg \max_{\boldsymbol{\theta}_2} \arg \min_{\boldsymbol{\theta}_1} \mathcal{L}^{\mathcal{D}_{train}}(\boldsymbol{\theta}; \mathbf{m}^{t-1}), \quad (8)$$

Algorithm 2 Projection Operation $\mathcal{P}_{\mathcal{C}}(\mathbf{a})$

Input: Vector $\mathbf{a} \in \mathbb{R}^d$, target domain $\mathcal{C} = \{\mathbf{a} : 0 \leq \mathbf{a}_i \leq 1, \|\mathbf{a}\|_1 \leq C\}$.

Computation Steps:

- 1) Computing the auxiliary variable b satisfying
 $\mathbf{1}^\top [\min(1, \max(0, \mathbf{a} - b \cdot \mathbf{1}))] - C = 0$
- 2) Computing the auxiliary variable c satisfying
 $c \leftarrow \max(0, b)$
- 3) Update $\mathbf{a}$
 $\mathbf{a}^* \leftarrow \min(1, \max(0, \mathbf{a} - c \cdot \mathbf{1}))$

Output: $\mathcal{P}_{\mathcal{C}}(\mathbf{a}) = \mathbf{a}^*$.

293 where $\mathcal{L}^{\mathcal{D}_{train}}(\boldsymbol{\theta}; \mathbf{m}^{t-1})$ denotes the objective function (3) with $\mathbf{m}$. The lower-
294 level problem is solved by stochastic gradient descent ascent in Algorithm 1.
295 *Step 2: Computing probability $\mathbf{s}^t$ with given $\boldsymbol{\theta}^t$.* From the probabilis-
296 tic BCGAN in (7), the learning target changes from the discrete masks $\mathbf{m}$
297 into the continuous probability $\mathbf{s}$, which is efficiently updated by the policy
298 gradient estimator (Zhou et al., 2022)

$$\nabla_{\mathbf{s}} \Phi(\mathbf{s}, \boldsymbol{\theta}) = \mathbb{E}_{p(\mathbf{m}|\mathbf{s})} \mathcal{L}^{\mathcal{D}_{meta}}(\boldsymbol{\theta}^*(\mathbf{m})) \nabla_{\mathbf{s}} \ln p(\mathbf{m} | \mathbf{s}). \quad (9)$$

299 This computing procedure **involves** unbiased gradient estimation, **elimini-**
300 **ating** the computational burdens **associated with** the inverse of the Hessian
301 matrix or implicit differentiation (Zhou et al., 2022; Zhao et al., 2023).

302 Denote η^t as the step size for updating the probability variable $\mathbf{s}$ at the
303 t -th step. Given $\boldsymbol{\theta}^t$,

$$\mathbf{s}^t \leftarrow \mathcal{P}_{\mathcal{C}_s}(\mathbf{s}^{t-1} - \eta^t \mathcal{L}^{\mathcal{D}_{meta}}(\boldsymbol{\theta}^t) \nabla_{\mathbf{s}} \ln p(\mathbf{m}^{t-1} | \mathbf{s}^{t-1})), \quad (10)$$

304 where the projection $\mathcal{P}_{\mathcal{C}_s}(\mathbf{s})$ from $\mathbf{s}$ to the domain $\mathcal{C}_s$ is summarized in Algo-
305 rithm 2. Moreover, *Appendix D* states theoretical guarantees of the closed-
306 form solution w.r.t. (10).

307 *Step 3: Updating mask $\mathbf{m}^t$ with given $\mathbf{s}^t$.* For $\mathbf{m}^t = (m_1^t, \dots, m_{d_0}^t) \in$
308 $\mathbb{R}^{d_0}$ following $\mathbf{m}_i \sim \text{Bern}(\mathbf{s}_i)$, we obtain the new mask $\mathbf{m}^t$ according to $\mathbf{s}^t$.
309 Finally, the obtained mask is left for future training or prediction processes.

310 **Remark 3.** *The manifold fitting module ($F_{\mathcal{M}}$) is designed to tackle addi-*
311 *tive noise and pixel noise, as supported by relevant work on statistical theory*
312 *and practical experiments (Yao et al., 2023; Yao and Xia, 2019; Yao et al.,*

2024b). This module exhibits reliable and robust properties. To further eliminate noisy dimensions or channels from vectors or grayscale images, the probabilistic bilevel masking framework is designed to learn binary masks (for vector data) or binary matrices (for matrix data and gray images). Both techniques are essential for enabling robust manifold fitting, image denoising, and interpolation.

4. Theoretical Assessments

This section presents the generalization analysis for the upper-level meta-learner of BCGAN. Let T be the maximum number of iterations. Denote η^t, γ_1^t and γ_2^t as the stepsize for updating $\mathbf{s}, \boldsymbol{\theta}_1, \boldsymbol{\theta}_2$ at the t -th iteration, respectively. All proofs can be found in *Appendixes B* and *C*.

Given meta set $\mathcal{D}_{meta} = \{\mathbf{z}_i\}_{i=1}^{m_1} \sim \mathbb{D}_1^{m_1} = \{(x_i, y_i)\}_{i=1}^{m_1} \sim \mathbb{D}_1^{m_1}$ and training set $\mathcal{D}_{train} = \{\mathbf{z}_i\}_{i=1}^{m_2} = \{(x_i, y_i)\}_{i=1}^{m_1+m_2} \sim \mathbb{D}_2^{m_1+m_2}$ by independent sampling from distributions $\mathbb{D}_1$ and $\mathbb{D}_2$, respectively. Let $(\mathbf{s}, \boldsymbol{\theta}_1, \boldsymbol{\theta}_2)$ in (7) be estimated by a stochastic algorithm A with data $\mathcal{D}_{meta}, \mathcal{D}_{train}$, i.e. $A(\mathcal{D}_{meta}, \mathcal{D}_{train})$. Now we introduce the upper-level population risk w.r.t $\mathbb{D}_1$ and the empirical risk w.r.t $\mathcal{D}_{meta}$ defined respectively as

$$R(A(\mathcal{D}_{meta}, \mathcal{D}_{train})) := R(\mathbf{s}, \boldsymbol{\theta}_1, \boldsymbol{\theta}_2) = \mathbb{E}_{\mathbf{z} \sim \mathbb{D}_1}[\ell(\mathbf{s}, \boldsymbol{\theta}_1, \boldsymbol{\theta}_2; \mathbf{z})],$$

and

$$R_{\mathcal{D}_{meta}}(A(\mathcal{D}_{meta}, \mathcal{D}_{train})) := R_{\mathcal{D}_{meta}}(\mathbf{s}, \boldsymbol{\theta}_1, \boldsymbol{\theta}_2) = \frac{1}{m_1} \sum_{i=1}^{m_1} [\ell(\mathbf{s}, \boldsymbol{\theta}_1, \boldsymbol{\theta}_2; \mathbf{z}_i)],$$

where ℓ is the objective function stated in (5) and $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2$ are the inner model parameters given the outer model parameter $\mathbf{s}$.

Motivated by previous works (Bao et al., 2021; Pan et al., 2024; Zhang et al., 2024), the algorithmic capability of meta learner is evaluated by

$$R^*(A(\mathcal{D}_{meta}, \mathcal{D}_{train})) := R(\mathbf{s}_*, \boldsymbol{\theta}_1, \boldsymbol{\theta}_2) = \inf_{\mathbf{s} \in \mathcal{C}_s} R(\mathbf{s}, \boldsymbol{\theta}_1, \boldsymbol{\theta}_2),$$

and

$$\hat{R}_{\mathcal{D}_{meta}}(A(\mathcal{D}_{meta}, \mathcal{D}_{train})) := R_{\mathcal{D}_{meta}}(\hat{\mathbf{s}}, \boldsymbol{\theta}_1, \boldsymbol{\theta}_2) = \inf_{\mathbf{s} \in \mathcal{C}_s} R_{\mathcal{D}_{meta}}(\mathbf{s}, \boldsymbol{\theta}_1, \boldsymbol{\theta}_2),$$

where $R_{\mathcal{D}_{meta}}(\hat{\mathbf{s}}, \boldsymbol{\theta}_1, \boldsymbol{\theta}_2) \leq R_{\mathcal{D}_{meta}}(\mathbf{s}_*, \boldsymbol{\theta}_1, \boldsymbol{\theta}_2)$.

334 The excess risk $\mathbb{E}[R(A(\mathcal{D}_{meta}, \mathcal{D}_{train})) - R^*(A(\mathcal{D}_{meta}, \mathcal{D}_{train}))]$ with $\mathbb{E} :=$
 335 $\mathbb{E}_{A, \mathcal{D}_{meta}, \mathcal{D}_{train}}$ can be bounded by

$$\begin{aligned} & \underbrace{\mathbb{E}[R(A(\mathcal{D}_{meta}, \mathcal{D}_{train})) - R_{\mathcal{D}_{meta}}(A(\mathcal{D}_{meta}, \mathcal{D}_{train}))]}_{(I)} \\ & + \underbrace{\mathbb{E}[R_{\mathcal{D}_{meta}}(A(\mathcal{D}_{meta}, \mathcal{D}_{train})) - \hat{R}_{\mathcal{D}_{meta}}(A(\mathcal{D}_{meta}, \mathcal{D}_{train}))]}_{(II)}, \end{aligned} \quad (11)$$

336 where (I) is the generalization gap to measure the difference between the
 337 population risk and the empirical risk, and (II) is the optimization error to
 338 characterize the approximation ability of **the** computing algorithm.

339 Denote Θ_1 and Θ_2 as the parameter space for θ_1 and θ_2 , respectively.
 340 The following conditions for objective functions have been extensively used
 341 in convergence analysis for bilevel optimization (Ji et al., 2021; Liu et al.,
 342 2022) and stability-based generalization analysis for single-level optimization
 343 (Hardt et al., 2016).

344 **Assumption 1.** Loss function ℓ is jointly l -Lipschitz continuous w.r.t. $\mathbf{s}, \theta_1$
 345 and θ_2 , if there holds

$$|\ell(\mathbf{s}, \theta_1, \theta_2; \mathbf{z}) - \ell(\mathbf{s}', \theta'_1, \theta'_2; \mathbf{z})| \leq l \left\| \begin{array}{c} \mathbf{s} - \mathbf{s}' \\ \theta_1 - \theta'_1 \\ \theta_2 - \theta'_2 \end{array} \right\|_2$$

346 for any $(\mathbf{s}, \theta_1, \theta_2), (\mathbf{s}', \theta'_1, \theta'_2) \in \mathcal{C}_s \times \Theta_1 \times \Theta_2$ and $\mathbf{z} \sim \mathbb{D}_1$.

347 **Assumption 2.** Loss function ℓ is jointly L -smooth w.r.t. $\mathbf{s}, \theta_1$ and θ_2 , if
 348 there holds

$$\|\nabla \ell(\mathbf{s}, \theta_1, \theta_2; \mathbf{z}) - \nabla \ell(\mathbf{s}', \theta'_1, \theta'_2; \mathbf{z})\|_2 \leq L \left\| \begin{array}{c} \mathbf{s} - \mathbf{s}' \\ \theta_1 - \theta'_1 \\ \theta_2 - \theta'_2 \end{array} \right\|_2$$

349 for any $(\mathbf{s}, \theta_1, \theta_2), (\mathbf{s}', \theta'_1, \theta'_2) \in \mathcal{C}_s \times \Theta_1 \times \Theta_2$ and $\mathbf{z} \sim \mathbb{D}_1$.

350 Now we state the upper bounds for Algorithm 1 on its generalization error
 351 and excess risk.

352 **Theorem 1.** Let Assumptions 1, 2 be true, and let A be Algorithm 1 with T
 353 iterations. *Given* the non-increasing step size $\eta^t = \max\{\eta^t, \gamma_1^t, \gamma_2^t\}$ satisfying
 354 $\eta^t \leq \frac{c_1}{tL} \leq 1$ with a positive constant c_1 , we have

$$|\mathbb{E}[R(A(\mathcal{D}_{\text{meta}}, \mathcal{D}_{\text{train}})) - R_{\mathcal{D}_{\text{meta}}}(A(\mathcal{D}_{\text{meta}}, \mathcal{D}_{\text{train}}))]| \leq \begin{cases} \mathcal{O}\left(\frac{T}{m_1}\right), & \text{if } 0 < c_1 < 1/7 \\ \mathcal{O}\left(\frac{T \ln(T)}{m_1}\right), & \text{if } c_1 = 1/7 \\ \mathcal{O}\left(\frac{T^{7c_1}}{m_1}\right), & \text{if } c_1 > 1/7. \end{cases}$$

355 **Remark 4.** Theorem 1 indicates that *the smoothness, step sizes, and the*
 356 *number of iterations significantly influence the generalization performance of*
 357 *SSGDA*. Indeed, the generalization error of minimax problems has been in-
 358 vestigated in some existing literature (Lei et al., 2021; Farnia and Ozdaglar,
 359 2021; Ozdaglar et al., 2022). In contrast, Theorem 1 characterizes the gen-
 360 eralization capability of the meta learner for the bilevel minimax problem. *To*
 361 *the best of our knowledge, the generalization analysis of the minimax prob-*
 362 *lem equipped with a bilevel scheme (Bao et al., 2021; Shu et al., 2023; Zhang*
 363 *et al., 2024) has not been previously discussed.*

364 **Theorem 2.** Suppose that loss function ℓ is convex with respect to $\mathbf{s}$, the
 365 domain $\mathcal{C}_{\mathbf{s}}$ is bounded in $\mathbb{R}^{d_0}$, and $\mathbb{E}[\|\mathbf{s}^0 - \hat{\mathbf{s}}\|] \leq V$. Under the same as-
 366 sumptions of Theorem 1, there holds

$$\begin{aligned} & \mathbb{E}\left[R_{\mathcal{D}_{\text{meta}}}(A(\mathcal{D}_{\text{meta}}, \mathcal{D}_{\text{train}})) - \hat{R}_{\mathcal{D}_{\text{meta}}}(A(\mathcal{D}_{\text{meta}}, \mathcal{D}_{\text{train}}))\right] \\ & \leq \mathcal{O}\left(\frac{V}{\max\{\eta, \gamma_1, \gamma_2\}T} + \max\{\eta, \gamma_1, \gamma_2\}l^2\right). \end{aligned}$$

367 Moreover, we have

$$\mathbb{E}[R(A(\mathcal{D}_{\text{meta}}, \mathcal{D}_{\text{train}})) - R^*(A(\mathcal{D}_{\text{meta}}, \mathcal{D}_{\text{train}}))] \leq \mathcal{O}(1/\sqrt{m_1})$$

368 by setting the iteration number $T \asymp \sqrt{m_1}$ and the step size $\eta^t = \max\{\eta^t, \gamma_1^t, \gamma_2^t\} \leq$
 369 $\min\{\frac{c}{\sqrt{T}}, \frac{c_1}{t}\}$ with $c, c_1 \in (0, 1/7)$,

370 **Remark 5.** Theorem 2 illustrates that the excess risk of the SSGDA algo-
 371 rithm can achieve the approximation rate of $\mathcal{O}(1/\sqrt{m_1})$ under mild condi-
 372 tions on the loss function and iteration design.

Remark 6. *The complexity of BCGAN is analyzed across its core modules. The upper-level probabilistic bilevel optimization (SSGDA) brings $\mathcal{O}(T \cdot (p + L \cdot r))$ complexity with backpropagation, where T is the number of outer iterations, p stands for the ambient dimension ($\mathbf{m} \in \mathbb{R}^p$). L and r are relevant to the lower-level GAN (with L layers and r parameters per layer). The lower-level cycle-consistent GAN training has a complexity of $\mathcal{O}(T \cdot L \cdot r)$, as each iteration involves forward and backward passes for both generators and discriminators. Overall, BCGAN’s complexity reaches $\mathcal{O}(T \cdot (p + 2L \cdot r))$. Practically, training BCGAN on Circle data requires approximately 17 minutes in these repeated experiments. To accelerate training, the stochastic Nesterov gradients (Gong et al., 2024) could be utilized in future investigations to reduce computational overhead.*

5. Experimental Evaluations

This section validates the effectiveness of BCGAN on simulated and real-world data. All experiments are implemented in Python on a PC equipped with an Intel(R) Xeon(R) Platinum 8175M CPU and an NVIDIA RTX A6000 GPU. Appendix E presents the detailed descriptions for the experimental settings. The implementation code and data are available at <https://github.com/zx1ml/BCGAN>.

5.1. Baselines and Parameter Selection

Baselines. The empirical performance of the proposed BCGAN is compared with RealNVP (Dinh et al., 2017), MAF (Papamakarios et al., 2017), MADE (Germain et al., 2015), RoundTrip (Liu et al., 2021), DAE (Vincent et al., 2008), CycleGAN (Zhu et al., 2017), MFCGAN (Yao et al., 2024b) and MFS (Yao et al., 2023).

Inspired by (Yao et al., 2024b), DAE, CycleGAN, MFCGAN and BCGAN are implemented independently with the same network structure and SSGDA algorithm, sharing similar hyperparameters and corresponding tuning strategies. The generators G_y from the latent space of these GANs are fed with uniformly generated random samples as inputs (Yao et al., 2024b). For other baselines originally designed for distribution estimation, we adopt the recent normalizing flow implementation¹ and modify their released code to save the generated samples for evaluation.

¹See https://github.com/kamenbliznashki/normalizing_flows/tree/master.

Parameter Selection. Similar to (Yao et al., 2024b), this paper sets the number of iterations as 200, the total sample size as 10^4 , and the total epochs as 200 by default. The relevant step sizes η, γ_1, γ_2 and penalty coefficient λ are all tuned across $[10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}]$ with a linear decay per 50 iterations, which are shared for all compared methods. Additionally, Appendix E.3 states the experimental analysis for the varying C on the specific datasets. The pool size of the image buffer is set to 50 by default, which stores previously generated images of GANs. Tables 2 and 3 provide the quantitative assessments of the manifold fitting by calculating the distances from sample points to the latent manifold, where three primary metrics are employed, including the mean and standard deviation, 95th percentile distance, and the percentage of extreme samples beyond 3σ . Furthermore, we present a visual representation of the generated samples from each algorithm, utilizing the scatter plots in Figures 1, 4 and 5. For simplicity, only the corrupted scenarios are plotted in these figures.

5.2. Empirical Validations on Synthetic Data

5.2.1. Manifold Estimation and Denoising on Synthetic Data

Denote $\mathcal{U}$ and $\mathcal{N}$ as the uniform distribution and the normal distribution. $\mathcal{N}(0, I_2)$ stands for the two-dimensional standard Gaussian distribution where I_i is a $i \times i$ identity matrix. Here, we first generate the following simulated datasets, which are used in (Yao et al., 2024a).

- *Case 1: Circle.* Generate $a_i \stackrel{i.i.d.}{\sim} \mathcal{U}(0, 1)$ and $\mathbf{b}_i \stackrel{i.i.d.}{\sim} \mathcal{N}(0, I_2) \in \mathbb{R}^2$ for $i = 1, 2, \dots, N$. Then the ambient data $\{X_i\}_{i=1}^N$ is collected by

$$X_i = (\cos(2\pi a_i), \sin(2\pi a_i)) + 0.01\mathbf{b}_i, \quad 1 \leq i \leq N. \quad (12)$$

Besides, we also consider adding one or more i.i.d. noisy dimensions, e.g., $X_{i,3} \in \mathcal{N}(0, 100)$. The same operation is also applied to the following datasets.

- *Case 2: Involute.* Generate $a_i \stackrel{i.i.d.}{\sim} \mathcal{U}(0, 1)$ and $\mathbf{b}_i \stackrel{i.i.d.}{\sim} \mathcal{N}(0, I_2) \in \mathbb{R}^2$ for $i = 1, 2, \dots, N$. The ambient data $\{X_i\}_{i=1}^N$ is collected by

$$X_i = (a_i \cos(6\pi a_i), a_i \sin(6\pi a_i)) + 0.01\mathbf{b}_i, \quad 1 \leq i \leq N. \quad (13)$$

- *Case 3: Torus.* Generate $a_i \stackrel{i.i.d}{\sim} \text{U}(0, 1)$, $b_i \stackrel{i.i.d}{\sim} \text{U}(0, 1)$ and $\mathbf{c}_i \sim \mathcal{N}(0, I_3) \in \mathbb{R}^3$ for $i = 1, 2, \dots, N$. The ambient data $\{X_i\}_{i=1}^N$ can be represented by $\{(X_{i,1}, X_{i,2}, X_{i,3}) + \mathbf{c}_i\}_{i=1}^N$ with

$$\begin{cases} X_{i,1} = (1 + \cos(2\pi a_i)/2) \cos(2\pi b_i) \\ X_{i,2} = (1 + \cos(2\pi a_i)/2) \sin(2\pi b_i) \\ X_{i,3} = \sin(2\pi a_i)/2 \end{cases} \quad (14)$$

- *Case 4: Calabi-Yau Manifold.* A classical illustration of the Calabi-Yau manifold is provided by the Fermat quartic $x_1^4 + x_2^4 + x_3^4 + x_4^4 = 0$, where (x_1, x_2, x_3, x_4) lies in the complex projective **three-dimensional** space. After the simplification operation of dividing by x_4^4 while constraining x_3^4/x_4^4 as a constant (Yao et al., 2024b), the Calabi-Yau manifold with three complex dimensions can be generated by

$$x_1^4 + x_2^4 = 1, \quad x_1, x_2 \in \mathbb{C}, \quad (15)$$

where the first component $x_1(a, b, c_1) = \exp\left(\frac{c_1\pi}{2}i\right) \sqrt{\cosh(a + bi)}$ and second component $x_2(a, b, c_2) = \exp\left(\frac{c_2\pi}{2}i\right) \sqrt{\sinh(b - ai)}$. $\text{Re}(a)$ and $\text{Im}(a)$ represent the real and imaginary parts of complex number a respectively. Each integer pair (c_1, c_2) satisfies $0 \leq c_1, c_2 \leq 3$, $a \in (-1.50, -1.45, -1.40, \dots, 1.50)$ and b encompasses a sequence from 0 to $\pi/2$ with step size of $1/640$. The resulting surface can be embedded in a 4-dimensional Euclidean space, by dividing (x_1, x_2) into their real and imaginary parts $(\text{Re}(x_1), \text{Re}(x_2), \text{Im}(x_1), \text{Im}(x_2)) \in \mathbb{R}^4$. As illustrated in (Yao et al., 2024b, 2023), the embedded manifold can be further projected to $\mathbb{R}^3$ with

$$(\text{Re}(x_1), \text{Re}(x_2), \cos(\alpha) \text{Im}(x_1) + \sin(\alpha) \text{Im}(x_2)),$$

which takes $\alpha = 1$ in practice for simplicity. Furthermore, the additive noise following $\mathcal{N}(0, 0.01)$ is added to the input in $\mathbb{R}^4$.

Additional details on collected data can be found in *Appendix E*. Based on the above generation steps, the ambient data $\mathcal{X}$ is collected with the default size of 313,296 for the Calabi-Yau manifold and 10^4 for the rest of the cases. For the target ambient data $\mathcal{Y}$, we also generate 10^4 points with each dimension subjected to $\text{U}(0, 1)$.

Table 2: Average distances between samples and latent manifold on three clean synthetic datasets. “Input” represents the information of the original training sets (the standard deviation $\sigma = 0.01$).

Model	Circle			Involute			Torus		
	Mean $\pm$ SD	95% quantile	Percent $> 3\sigma$	Mean $\pm$ SD	95% quantile	Percent $> 3\sigma$	Mean $\pm$ SD	95% quantile	Percent $> 3\sigma$
Input	$8.13(10^{-3}) \pm 6.14(10^{-3})$	$2.00(10^{-2})$	0.42%	$8.29(10^{-3}) \pm 5.90(10^{-3})$	$1.97(10^{-2})$	0.33%	$7.98(10^{-3}) \pm 6.04(10^{-3})$	$1.98(10^{-2})$	0.22%
RealNVP	$3.17(10^{-2}) \pm 1.07(10^{-1})$	$1.29(10^{-1})$	12.80%	$6.89(10^{-2}) \pm 1.11(10^{-1})$	$15.61(10^{-2})$	60.77%	$8.15(10^{-2}) \pm 2.02(10^{-1})$	$3.22(10^{-1})$	45.90%
MAF	$3.72(10^{-2}) \pm 8.54(10^{-2})$	$1.86(10^{-1})$	21.19%	$6.85(10^{-2}) \pm 5.18(10^{-2})$	$1.53(10^{-1})$	69.94%	$9.05(10^{-2}) \pm 1.06(10^{-1})$	$3.23(10^{-1})$	58.93%
MADE	$3.67(10^{-1}) \pm 3.01(10^{-1})$	$8.81(10^{-1})$	93.59%	$8.99(10^{-2}) \pm 7.89(10^{-2})$	$1.65(10^{-1})$	80.21%	$1.89(10^{-1}) \pm 1.79(10^{-1})$	$4.77(10^{-1})$	87.78%
RoundTrip	$1.78(10^{-2}) \pm 8.06(10^{-3})$	$3.22(10^{-2})$	12.10%	$2.09(10^{-2}) \pm 2.34(10^{-2})$	$7.59(10^{-2})$	15.33%	$1.65(10^{-2}) \pm 1.21(10^{-2})$	$4.02(10^{-2})$	13.14%
DAE	$1.26(10^{-3}) \pm 1.02(10^{-3})$	$3.29(10^{-3})$	0.00%	$3.21(10^{-2}) \pm 5.44(10^{-2})$	$1.53(10^{-1})$	23.51%	$2.31(10^{-2}) \pm 3.29(10^{-2})$	$7.37(10^{-2})$	2.41%
CycleGAN	$1.95(10^{-3}) \pm 1.44(10^{-3})$	$3.65(10^{-3})$	0.00%	$5.86(10^{-3}) \pm 1.14(10^{-2})$	$1.44(10^{-2})$	2.47%	$3.79(10^{-3}) \pm 1.28(10^{-2})$	$7.84(10^{-1})$	2.19%
MFCGAN	$7.45(10^{-4}) \pm 8.47(10^{-4})$	$2.34(10^{-3})$	0.00%	$5.13(10^{-3}) \pm 3.26(10^{-2})$	$1.16(10^{-2})$	2.31%	$6.81(10^{-3}) \pm 2.73(10^{-2})$	$1.05(10^{-2})$	1.81%
BCGAN (Ours)	$7.21(10^{-4}) \pm 7.61(10^{-4})$	$2.17(10^{-3})$	0.00%	$5.08(10^{-3}) \pm 3.19(10^{-2})$	$1.18(10^{-2})$	2.18%	$6.37(10^{-3}) \pm 2.59(10^{-2})$	$9.62(10^{-3})$	1.74%

Table 3: Average distances between samples and latent manifold on three corrupted synthetic datasets. These results are obtained on corrupted data with noisy dimensions following $\mathcal{N}(0, 100)$.

Model	Circle with corrupted input			Involute with corrupted input			Torus with corrupted output		
	Mean $\pm$ SD	95% quantile	Percent $> 3\sigma$	Mean $\pm$ SD	95% quantile	Percent $> 3\sigma$	Mean $\pm$ SD	95% quantile	Percent $> 3\sigma$
Input	$7.06(10^0) \pm 5.88(10^0)$	$18.62(10^0)$	98.04%	$7.85(10^0) \pm 6.01(10^0)$	$19.51(10^0)$	99.76%	$7.98(10^{-3}) \pm 6.04(10^{-3})$	$1.98(10^{-2})$	0.22%
RealNVP	$8.15(10^0) \pm 8.68(10^0)$	$22.98(10^0)$	98.06%	$9.46(10^0) \pm 6.93(10^0)$	$22.14(10^0)$	99.94%	$8.36(10^{-2}) \pm 1.57(10^{-1})$	$3.26(10^{-1})$	46.99%
MAF	$9.10(10^0) \pm 8.98(10^0)$	$2.48(10^2)$	98.85%	$9.44(10^0) \pm 6.81(10^0)$	$21.65(10^0)$	99.96%	$8.56(10^{-2}) \pm 1.04(10^{-1})$	$3.13(10^{-1})$	56.31%
MADE	$7.56(10^0) \pm 7.97(10^0)$	$21.69(10^0)$	99.33%	$7.92(10^0) \pm 5.92(10^0)$	$19.23(10^0)$	99.95%	$1.89(10^{-1}) \pm 1.79(10^{-1})$	$4.77(10^{-1})$	87.75%
RoundTrip	$7.44(10^0) \pm 6.06(10^0)$	$19.32(10^0)$	99.82%	$10.35(10^0) \pm 7.53(10^0)$	$24.71(10^0)$	100.00%	$4.72(10^{-2}) \pm 4.56(10^{-2})$	$1.31(10^{-1})$	56.84%
DAE	$5.42(10^{-1}) \pm 4.46(10^{-1})$	$7.37(10^{-1})$	21.74%	$8.47(10^{-1}) \pm 6.52(10^{-1})$	$15.42(10^{-1})$	37.41%	$3.40(10^{-2}) \pm 5.33(10^{-2})$	$8.79(10^{-2})$	35.94%
CycleGAN	$7.54(10^0) \pm 6.32(10^0)$	$20.82(10^0)$	98.43%	$7.71(10^0) \pm 4.09(10^0)$	$16.11(10^0)$	99.81%	$3.86(10^{-2}) \pm 4.62(10^{-2})$	$9.75(10^{-2})$	41.13%
MFCGAN	$7.54(10^0) \pm 6.32(10^0)$	$20.82(10^0)$	99.63%	$7.22(10^0) \pm 4.55(10^0)$	$14.78(10^0)$	99.77%	$3.27(10^{-2}) \pm 5.24(10^{-2})$	$8.36(10^{-2})$	34.79%
BCGAN (Ours)	$7.49(10^{-4}) \pm 8.02(10^{-4})$	$2.26(10^{-3})$	0.00%	$5.17(10^{-3}) \pm 3.25(10^{-2})$	$1.27(10^{-2})$	2.53%	$6.44(10^{-3}) \pm 2.71(10^{-2})$	$1.06(10^{-2})$	2.09%

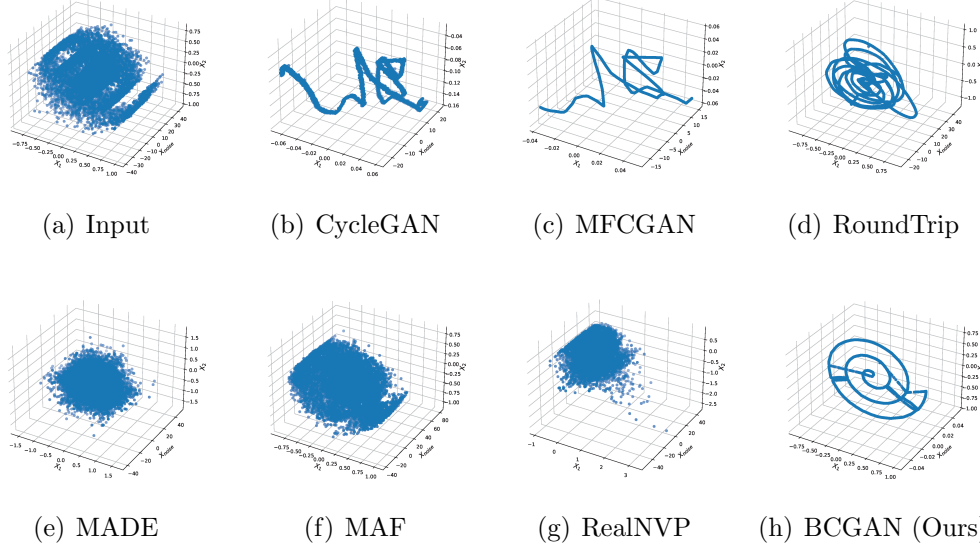


Figure 4: Results in the Involute dataset with noisy input. Clean dataset includes input $[X_1, X_2]$, while the corrupted dataset involves $[X_1, X_2, X_{noise}]$ with $X_{noise} \in \mathcal{N}(0, 100)$.

Experimental evaluations are conducted under both clean and noisy scenarios. In Figure 1 on the circle dataset, BCGAN demonstrates superior performance in reconstructing the underlying manifold structure from noisy inputs, whereas baseline methods like CycleGAN, MFCGAN, and MADE struggle with noise-induced distortions. This result highlights BCGAN’s ability to automatically identify and mask noisy dimensions, allowing for accurate manifold estimation even in the presence of significant data corruption.

Table 2 shows that our BCGAN provides better approximation performance on clean datasets, where distances between sample points and the latent manifold are much smaller than other baselines. To better validate the robustness with corrupted input x or output y , we consider adding noisy dimension $X_{noise} \in \mathcal{N}(0, 100)$ to the input of circle and involute datasets, as well as $Y_{noise} \in \mathcal{N}(0, 100)$ in the output of torus dataset. As presented in Figures 1 and 4, extreme random corruptions in the input obviously mislead these baselines to inappropriate mappings or density estimation, and evidently degrade their fitting capability. This is reasonable since CycleGAN and MFCGAN utilize the cycle consistency loss (1). Other baselines are ini-

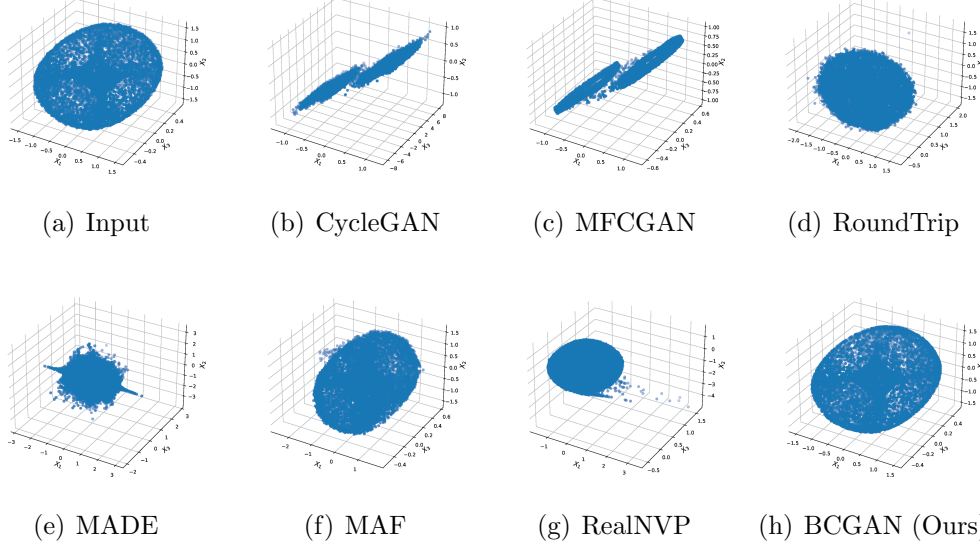


Figure 5: Results in Torus dataset with noisy output. Clean Involute dataset includes output $[Y]$, while the corrupted dataset involves $[Y, Y_{noise}]$ with $Y_{noise} \in \mathcal{N}(0, 100)$.

478 tially designed for density estimation, which heavily relies on the quality of
 479 the original inputs. Our findings on BCGAN are consistent with the obser-
 480 vation in (Yao et al., 2024a) that operations on the original data space **can**
 481 **help to reduce the** information loss.

482 Table 3 and Figure 5 further quantify the robustness of BCGAN and some
 483 unsupervised baselines (RealNVP, MAF, and MADE) against noisy inputs or
 484 outputs. **Our findings verify that** BCGAN is robust against noisy corruptions,
 485 whereas random information **negatively impacts** the density estimation or the
 486 Euclidean distance in the cycle consistency loss. Notably, the fitting results
 487 on the torus show that the noisy outputs are also harmful to CycleGAN
 488 and MFCGAN for manifold fitting and supervised RoundTrip for density
 489 estimation. These simulated results demonstrate that the proposed BCGAN
 490 could assign the proper masks on y and present the lowest fitting error.

491 Moreover, we evaluate the performance of BCGAN for the Calabi-Yau
 492 manifold with noisy features (Yau, 2008). Here, the original input is cor-
 493 rupted by a new dimension $X_{noise} \in \mathcal{N}(0, 100)$. In Figure 6, we visualize the
 494 results by comparing the generated points and the underlying manifold. We
 495 also adopt the pseudo-bias defined as the norm of $x_1^4 + x_2^4 - 1$ for $x \in \mathbb{C}^2$ (Yao

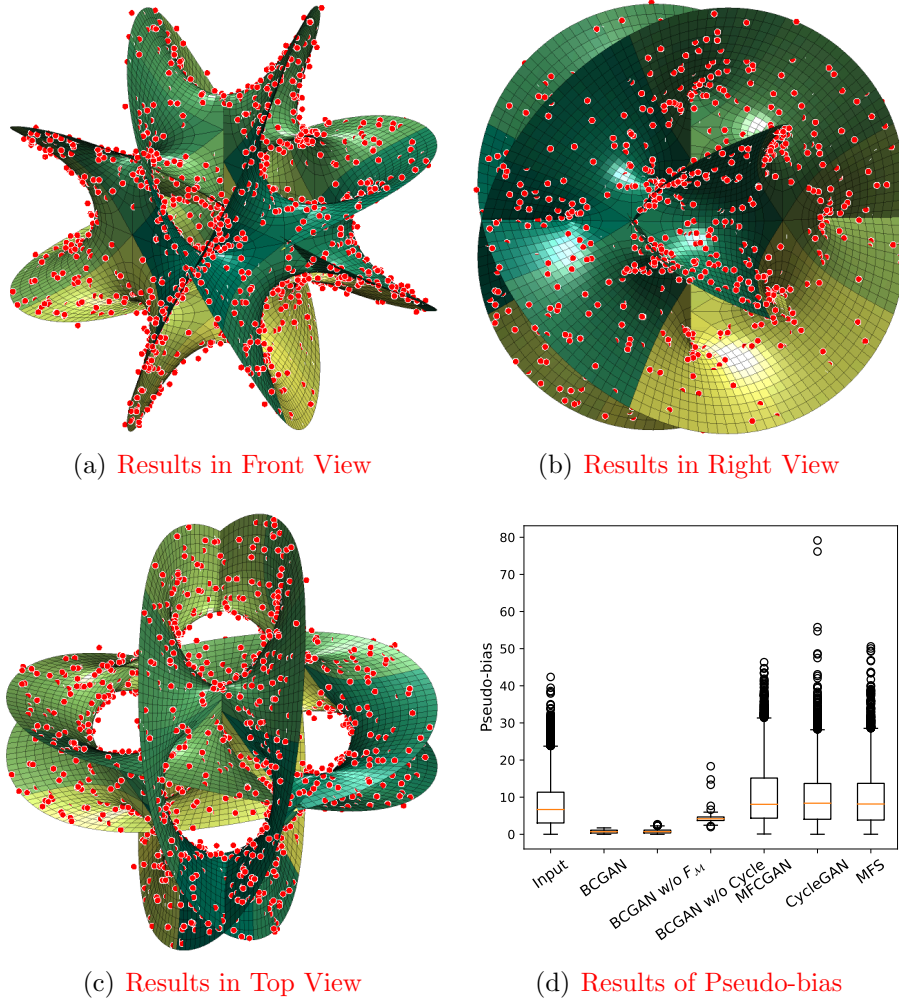


Figure 6: Experiments on Calabi-Yau manifold with noisy input $X_{noise} \in \mathcal{N}(0, 100)$. **Figures (a) to (c) show different views of the fitting results on the surfaces of the 3D projected manifold, where the red points represent the generated samples.** Figure (d) presents the pseudo-bias between input and generated samples of these baselines. Furthermore, ablation experiments **are conducted** by removing module F_M for completeness.

et al., 2024b), with a boxplot in Figure 6. As validated in (Yao et al., 2024b),
MFCGAN is fully capable of learning from data with merely additive noise.
However, the results in the boxplot **reveal** its sensitivity to corruptions **such**
as noisy dimensions and channels. In contrast, BCGAN provides the narrow-
est pseudo-bias distribution even with noisy ambient representation, which

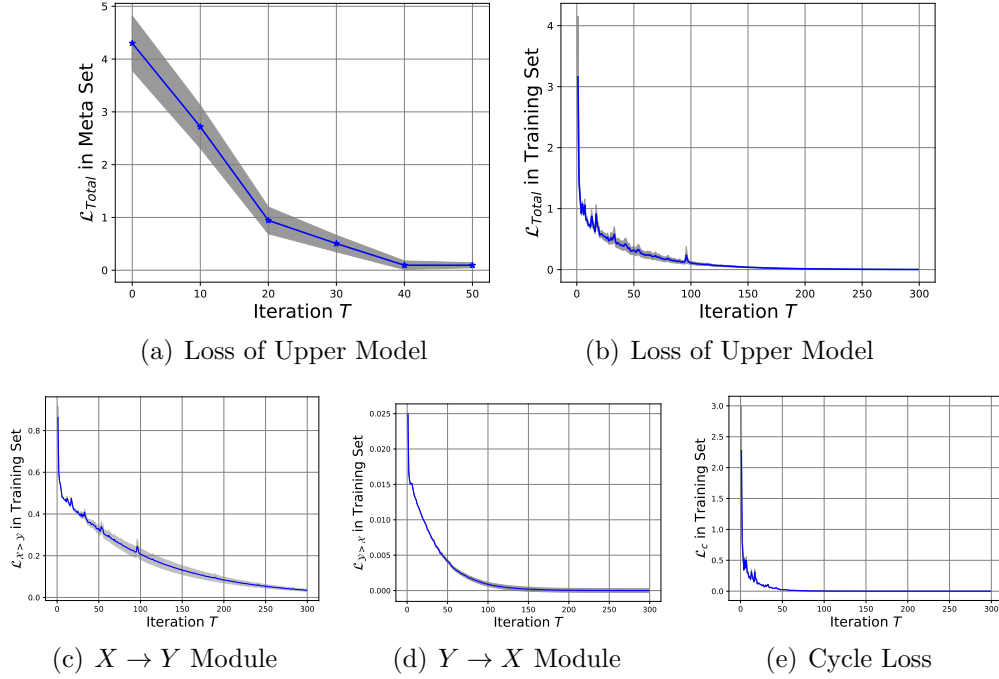


Figure 7: Convergence validation on circle dataset. Figures (a) and (b) present the curves of the upper and lower objective functions. (c)-(e) correspond to three components of the lower-level objective function, respectively.

501 verifies its robust under complex noise scenarios. The ablation studies are
 502 conducted by removing the manifold fitting module $\mathcal{F}$ and regarding MFC-
 503 GAN as the baseline without the bilevel masking mechanism. The results
 504 in Figure 6 demonstrate that the manifold fitting module $F_{\mathcal{M}}$ is robust to
 505 additive noise, and the bilevel mask mechanism is effective in handling noisy
 506 channels and dimensions.

507 5.2.2. Validation on Optimization Convergence

508 In the following, we validate the convergence performance of the meta-
 509 learner and the lower predictor in BCGAN. For simplicity, we employ the
 510 SSGDA algorithm and conduct experiments on the circle fitting task, pre-
 511 senting the training and validation losses. The step sizes of the upper and
 512 lower models are set to be inversely proportional to the number of iterations.

513 Figure 7(a) shows that the upper-level optimization can reach conver-
 514 gence, where $\mathcal{L}_{Total} := \mathcal{L}_{X \rightarrow Y} + \mathcal{L}_{Y \rightarrow X} + \lambda \mathcal{L}_c$ as well as its variance on the

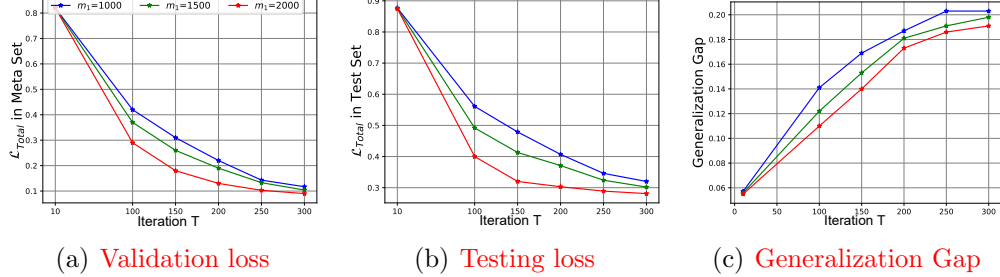


Figure 8: Generalization validation in circle fitting task with varying sample size m_1 and maximum iterations T .

validation set quickly decrease and then stabilize within 50 iterations. Moreover, Figure 7(b) shows the lower-level convergence curves at the last outer iteration, indicating that the proper masks have been assigned to the targets. Figures 7(c)-7(e) verify that three loss counterparts on the training sets tend to zero with the learned masks.

5.2.3. Validation on Algorithmic Generalization

Theorem 1 establishes that the generalization gap for the SSGDA algorithm is governed by the validation sample size m_1 , the number of iterations T , and step-size configurations. Consistent with prior work (Ren et al., 2018; Bao et al., 2021; Zhang et al., 2024), we assume the meta and test distributions are identical, although they may diverge from the training distribution. We assess the generalization capability of the outer-level algorithm, as defined in Eq.(11), by quantifying the discrepancy between validation and test errors (Bao et al., 2021; Zhang et al., 2024). For simplicity, we present empirical results for the circle fitting task using Algorithm 1, with sample size $m_1 \in [1000, 1500, 2000]$, and maximum iterations $T \in [10, 100, 150, 200, 250, 300]$.

Moreover, the empirical findings reveal some interesting conclusions that align with our theory and offer practical suggestions.

Impact of sample size m_1 . Figure 8 illustrates the performance of the SSGDA algorithm under varying values of T and m_1 . Notably, the results indicate that a larger meta set accelerates optimization convergence and enhances generalization, corroborating Theorem 1. Conversely, the algorithm faces significant limitations under conditions of data scarcity. A limited sample size m_1 may result in non-unique solutions and unstable outputs, con-

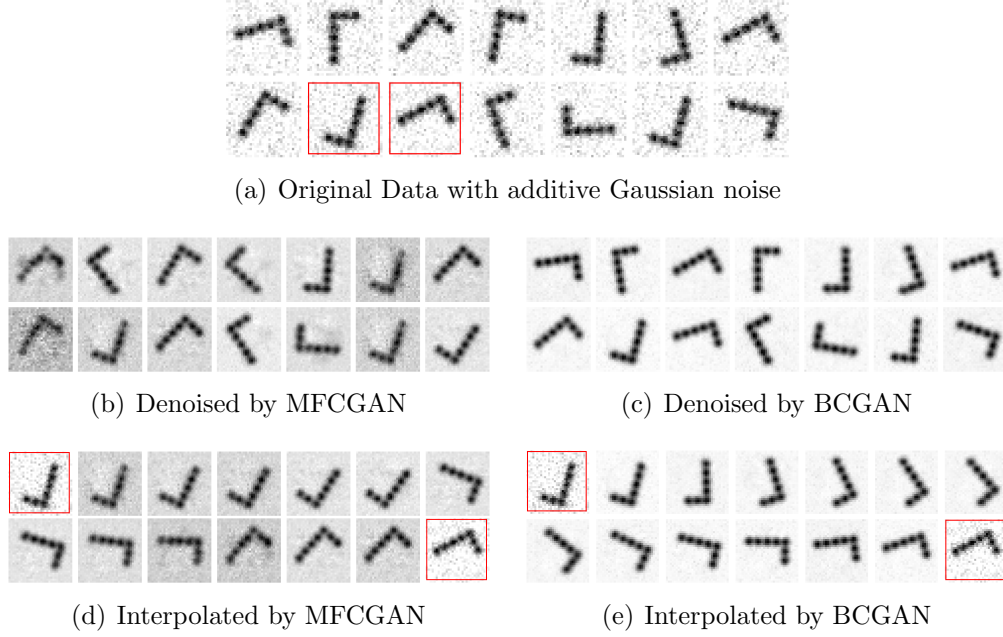


Figure 9: Experiments on “L”-shape graphs corrupted by white noise and noisy output. (a) presents the graphs after different rotation angles, where two objects with red boxes are selected for interpolation. (b) and (c) show the recovered results on denoising. (d) and (e) state the interpolated graphs.

sequently impairing generalization. Furthermore, as detailed in Theorem 2, an insufficient number of samples can prevent the algorithm from converging effectively, resulting in a significant excess risk.

Theoretical and empirical insights. The synthesis of theoretical analysis and empirical assessments provides insights into the generalization capabilities of the proposed BCGAN and a broader class of single-time scale bilevel optimization algorithms. Specifically, its generalization performance is often improved with an increase in the size of the meta set m_1 and judiciously chosen iteration counts T . Insufficient iterations may precipitate underfitting, while an excess can incline towards overfitting. Typically, establishing appropriate learning rates is instrumental for generalization.

5.3. Manifold Denoising and Interpolating in Images

This subsection provides experimental results on image datasets (i.e., rotated “L”-shape images, and the frames of Charlie Chaplin’s walking and

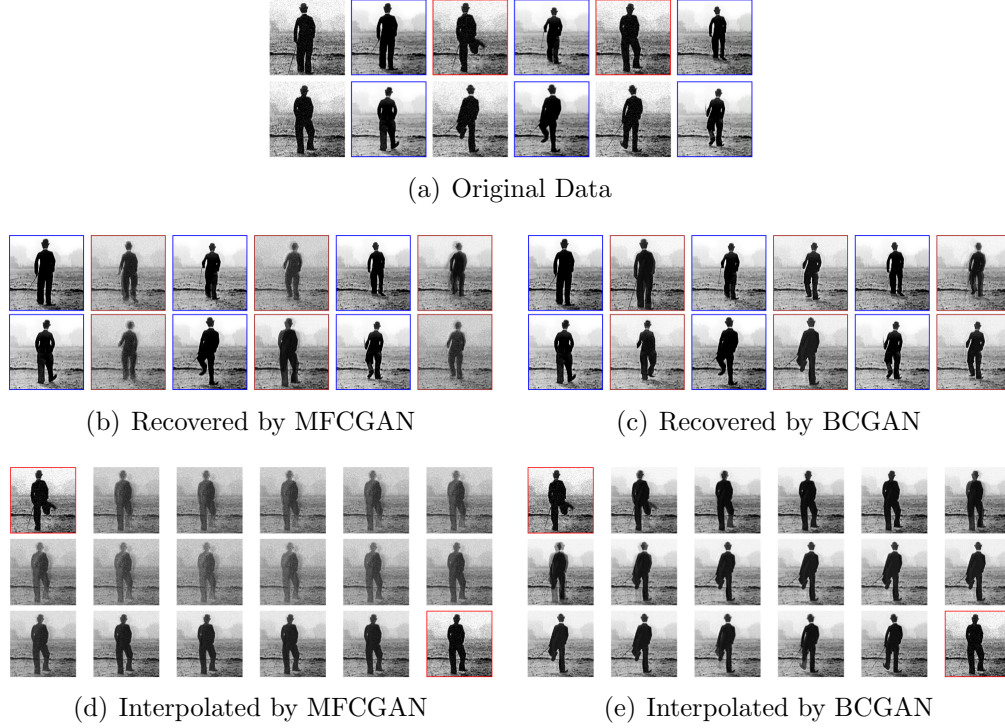


Figure 10: Experiments on Charlie Chaplin’s frames with inputs corrupted by white noise and outputs involving noisy dimension following $\mathcal{N}(0,100)$. (a) presents the bitmaps extracted from a GIF. Graphs with blue boxes are the test samples without adding white noise, whereas the two objects with red boxes are selected for interpolation. (b) and (c) show the generated new graphs in brown boxes when feeding unseen test set to $\hat{G}_{\mathcal{X}}$ and $\hat{G}_{\mathcal{Y}}$. (d) and (e) correspond to the interpolated observations.

554 **Chow Yun-Fat’ Clapping (Yao et al., 2024b)) to demonstrate** the applicabil-
 555 ity of our proposal **for** denoising and nonlinear interpolation.

556 *Rotated “L” images.* The first foundational image is generated in ac-
 557 cordance with the density function of a Gaussian mixture model, which is
 558 configured to simulate an “L”-shape graph. We rotated the image at inter-
 559 vals of 10 degrees and replicated it 15 times by adding white noise $\mathcal{N}(0, 0.01)$,
 560 resulting in a total of 540 images with the size of 32×32 . The noisy graphs
 561 under different rotation degrees are regarded as inputs (see Figure 9(a)).
 562 Here, a new dimension $Y_{noise} \in \mathcal{N}(0, 100)$ to y to validate the robustness
 563 against label noise.

564 Figure 9(b) shows that the noisy channels or dimensions in the output

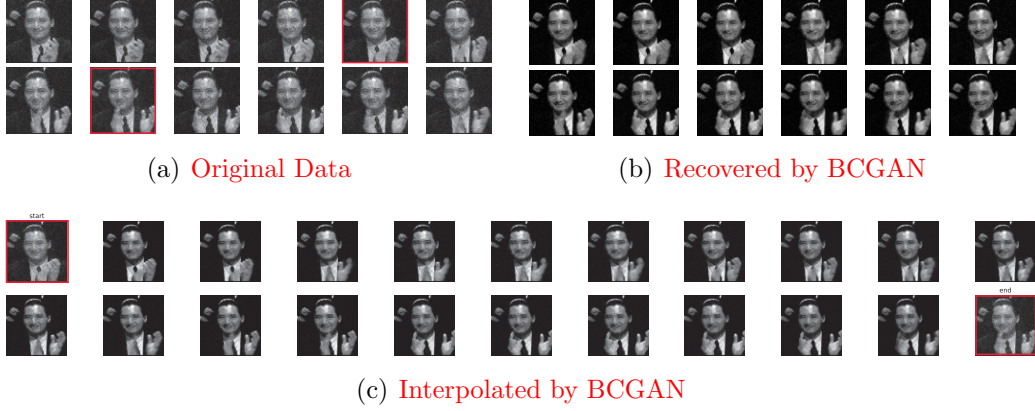


Figure 11: Experiments on Chow Yun-Fat’s Clapping frames with original inputs. (a) presents the bitmaps extracted from a GIF. Graphs with red boxes are selected for interpolation. (b) shows the denoised images recovered by BCGAN. (c) corresponds to the interpolated observations.

degrade the fitting and alignment among the ambient and latent spaces for MFCGAN. One can easily find that the rotation degrees of some images processed by MFCGAN are inconsistent with the original ones. BCGAN can effectively remove the noisy components in the inputs and outputs, while retaining the original rotations as much as possible, as shown in Figure 9(c). Furthermore, the nonlinear interpolation is assessed by continuously generating samples between the two selected graphs, highlighted by red boxes, whose inputs are denoted by x_i and x_j . The next step is to compute $t\hat{G}_{\mathcal{X}}(x_i) + (1-t)\hat{G}_{\mathcal{X}}(x_j)$ for different t , which are further fed into $\hat{G}_{\mathcal{Y}}$ to generate a series of intermediate images. As shown in Figure 9(d), the images generated by MFCGAN still keep distinct noisy pixels with incoherent rotation angles and might arrange the images in a sequence that diverges from their original rotation orders. Interestingly, during the interpolation process, BCGAN generates some new graphs that can not be found in the training set. Even under noisy scenarios, BCGAN is capable of simultaneously achieving efficient noise reduction and accurate nonlinear interpolation, as shown in Figures 9(c) and 9(e).

Frames of Charlie Chaplin’s Walking. We conducted experiments on the frames extracted from a short GIF of Charlie Chaplin. Here, 34 frames of black and white graphs with the size of 160×160 are selected, where each frame with an odd index is augmented by adding white noise $\mathcal{N}(0, 0.01)$ 20



(a) Noisy (left) and Recovered (right) Data with JPEG Compression Artifacts



(b) Noisy (left) and Recovered (right) Data with Gamma Noise



(c) Noisy (left) and Recovered (right) Data with Salt & Pepper Noise



(d) Noisy (left) and Recovered (right) Data with White Noise

Figure 12: Experiments on Chow Yun-Fat’s Clapping frames with various types of noisy inputs, and outputs involving noisy dimension following $\mathcal{N}(0, 100)$. (a)-(e) present corrupted inputs (in left panels) and the denoised results (in right panels) by BCGAN under the noise types of JPEG Compression Artifacts, Gamma Noise, Salt & Pepper Noise and White Noise, respectively.

586 times. These odd-indexed frames are collected as the training set, while the
 587 rest are left for testing. A slice of generated samples is shown in Figure 10(a),
 588 where two training samples with red boxes are selected for interpolation, and
 589 the testing set is highlighted with blue boxes.

590 To better evaluate the generalization performance of BCGAN, we feed
 591 testing samples into $\hat{G}_x$ and $\hat{G}_y$ to generate corresponding manifold embed-
 592 dings and new interpolated samples, respectively. Figures 10(b) and 10(c)
 593 verify that BCGAN can generalize to unseen data, learn the mappings be-
 594 tween two spaces, and assign the proper masks on the target variables. The

series of recovered samples produced by BCGAN is visually closer to the original testing samples than those processed by MFCGAN. Moreover, the interpolated images by BCGAN in Figure 10(e) demonstrate the coherent movements in the limbs and the angles of the cane compared with MFCGAN.

Frames of Chow Yun-Fat’s Clapping. To assess the capability of image denoising and interpolation, we conducted experiments on the frames of Chow Yun-Fat’s clapping, extracting 280 gray frames (64×64) and using 12 for testing. As illustrated in Figure 11, BCGAN demonstrates superior performance in both denoising and nonlinear interpolation. The model effectively suppresses background noise and generates smooth transitions between selected anchor frames, which indicates an accurate estimation of the underlying manifold geometry.

Furthermore, we validate the robustness of BCGAN by subjecting the whole dataset to five distinct types of noise, including JPEG compression artifacts, Gamma, Salt & Pepper, and White noise. As shown in Figure 12, BCGAN achieves high-fidelity image recovery across all types of corruption, successfully eliminating diverse noise artifacts while preserving critical edge and moving details. These results confirm that the bilevel masking mechanism successfully mitigates pixel-level input noise and noisy dimensions in the output variables, thereby ensuring high-quality reconstruction under complex data degradation.

6. Conclusion

This paper **presents** a new manifold fitting scheme called BCGAN, which **incorporates** feature selection into a probabilistic bilevel optimization framework. Compared with the existing works, the proposed approach is capable of realizing dimensional reduction, robust image recovery, and nonlinear interpolation simultaneously. Theoretical guarantees are presented for its generalization bounds, and empirical evaluations are stated to verify its superiority over the related state-of-the-art models.

Acknowledgments

The work is supported in part by the National Natural Science Foundation of China (Grant Nos 62376104 and 12426512).

628 References

- 629 Antoine, J.P., Roşca, D., Vandergheynst, P., 2010. Wavelet transform on
630 manifolds: old and new approaches. *Applied and Computational Harmonic*
631 *Analysis* 28, 189–202.
- 632 Bao, F., Wu, G., Li, C., Zhu, J., Zhang, B., 2021. Stability and generalization
633 of bilevel programming in hyperparameter optimization, in: *Advances in*
634 *Neural Information Processing Systems*, pp. 4529–4541.
- 635 Borsos, Z., Mutny, M., Krause, A., 2020. Coresets via bilevel optimization
636 for continual learning and streaming. *Advances in Neural Information*
637 *Processing Systems* 33, 14879–14890.
- 638 Chakraborty, R., Bouza, J., Manton, J.H., Vemuri, B.C., 2020. Manifoldnet:
639 A deep neural network for manifold-valued data with applications. *IEEE*
640 *Transactions on Pattern Analysis and Machine Intelligence* 44, 799–810.
- 641 Coifman, R.R., Lafon, S., 2006. Diffusion maps. *Applied and computational*
642 *harmonic analysis* 21, 5–30.
- 643 Deutsch, S., Ortega, A., Medioni, G., 2016. Manifold denoising based on spec-
644 tral graph wavelets. *IEEE International Conference on Acoustics, Speech*
645 *and Signal Processing* , 4673–4677.
- 646 Dinh, L., Sohl-Dickstein, J., Bengio, S., 2017. Density estimation using real
647 nvp, in: *International Conference on Learning Representations*.
- 648 Farnia, F., Ozdaglar, A., 2021. Train simultaneously, generalize better: Sta-
649 bility of gradient-based minimax learners, in: *International Conference on*
650 *Machine Learning*, pp. 3174–3185.
- 651 Fefferman, C., Ivanov, S., Kurylev, Y., Lassas, M., Narayanan, H., 2018.
652 Fitting a putative manifold to noisy data, in: *Conference On Learning*
653 *Theory*, pp. 688–720.
- 654 Fefferman, C., Ivanov, S., Kurylev, Y., Lassas, M., Narayanan, H., 2020.
655 Reconstruction and interpolation of manifolds. i: The geometric whitney
656 problem. *Foundations of Computational Mathematics* 20, 1035–1133.

- 657 Franceschi, L., Frasconi, P., Salzo, S., Grazzi, R., Pontil, M., 2018. Bilevel
658 programming for hyperparameter optimization and meta-learning, in: In-
659 ternational Conference on Machine Learning, pp. 1568–1577.
- 660 Germain, M., Gregor, K., Murray, I., Larochelle, H., 2015. Made: Masked
661 autoencoder for distribution estimation. International Conference on Ma-
662 chine Learning , 881–889.
- 663 Gong, X., Hao, J., Liu, M., 2024. An accelerated algorithm for stochastic
664 bilevel optimization under unbounded smoothness. Advances in Neural
665 Information Processing Systems 37, 78201–78243.
- 666 Gregory, K., Mammen, E., Wahl, M., 2021. Statistical inference in sparse
667 high-dimensional additive models. The Annals of Statistics 49, 1514–1536.
- 668 Hao, J., Ji, K., Liu, M., 2024. Bilevel coreset selection in continual learn-
669 ing: A new formulation and algorithm. Advances in Neural Information
670 Processing Systems .
- 671 Hardt, M., Recht, B., Singer, Y., 2016. Train faster, generalize better: Stabil-
672 ity of stochastic gradient descent, in: International Conference on Machine
673 Learning, pp. 1225–1234.
- 674 Harris, P., Brunson, C., Charlton, M., 2011. Geographically weighted prin-
675 cipal components analysis. International Journal of Geographical Informa-
676 tion Science 25, 1717–1736.
- 677 Hastie, T., Stuetzle, W., 1989. Principal curves. Journal of the American
678 statistical association 84, 502–516.
- 679 He, K., Chen, X., Xie, S., Li, Y., Dollár, P., Girshick, R., 2022. Masked au-
680 toencoders are scalable vision learners, in: Proceedings of the IEEE/CVF
681 conference on computer vision and pattern recognition, pp. 16000–16009.
- 682 Hein, M., Maier, M., 2006. Manifold denoising. Advances in Neural Infor-
683 mation Processing Systems .
- 684 Huang, Z., Wu, J., Van Gool, L., 2019. Manifold-valued image generation
685 with wasserstein generative adversarial nets, in: Proceedings of the AAAI
686 Conference on Artificial Intelligence, pp. 3886–3893.

687 Ji, K., Yang, J., Liang, Y., 2021. Bilevel optimization: Convergence analysis
688 and enhanced design. *International conference on machine learning* , 4882–
689 4892.

690 Lei, Y., Yang, Z., Yang, T., Ying, Y., 2021. Stability and generalization
691 of stochastic gradient methods for minimax problems, in: *International*
692 *Conference on Machine Learning*, pp. 6175–6186.

693 Lei, Y., Ying, Y., 2020. Fine-grained analysis of stability and generalization
694 for stochastic gradient descent, in: *International Conference on Machine*
695 *Learning*, pp. 5809–5819.

696 Liu, B., Ye, M., Wright, S., Stone, P., Liu, Q., 2022. Bome! bilevel opti-
697 mization made easy: A simple first-order approach. *Advances in Neural*
698 *Information Processing Systems* , 17248–17262.

699 Liu, Q., Xu, J., Jiang, R., Wong, W.H., 2021. Density estimation using
700 deep generative neural networks. *Proceedings of the National Academy of*
701 *Sciences* 118, e2101344118.

702 Luo, S., Hu, W., 2020. Differentiable manifold reconstruction for point cloud
703 denoising, in: *Proceedings of the 28th ACM International Conference on*
704 *Multimedia*, pp. 1330–1338.

705 Ma, C., Zhao, B., Chen, C., Rudin, C., 2023. This looks like those: Illu-
706 minating prototypical concepts using multiple visualizations. *Advances in*
707 *Neural Information Processing Systems* 36, 39212–39235.

708 Meilă, M., Zhang, H., 2023. Manifold learning: what, how, and why. *Annual*
709 *Review of Statistics and Its Application* 11, 39–417.

710 Miolane, N., Holmes, S., 2020. Learning weighted submanifolds with vari-
711 ational autoencoders and riemannian variational autoencoders, in: *Pro-*
712 *ceedings of the IEEE/CVF conference on computer vision and pattern*
713 *recognition*, pp. 14503–14511.

714 Mohammed, K., Narayanan, H., 2017. Manifold learning using kernel den-
715 sity estimation and local principal components analysis. *arXiv preprint*
716 *arXiv:1709.03615* .

- 717 Ozdaglar, A., Pattathil, S., Zhang, J., Zhang, K., 2022. What is a good
718 metric to study generalization of minimax learners? *Advances in Neural*
719 *Information Processing Systems* 35, 38190–38203.
- 720 Pan, X., Li, X., Liu, J., Sun, T., Sun, K., Chen, L., Qu, Z., 2024. Stabil-
721 ity and generalization for stochastic recursive momentum-based algorithms
722 for (Strongly-)Convex one to k -level stochastic optimizations, in: *International Conference on Machine Learning*, pp. 39201–39275.
- 724 Papaioannou, P.G., Talmon, R., Kevrekidis, I.G., Siettos, C., 2022. Time-
725 series forecasting using manifold learning, radial basis function interpola-
726 tion, and geometric harmonics. *Chaos: An Interdisciplinary Journal of*
727 *Nonlinear Science* 32, 083113.
- 728 Papamakarios, G., Pavlakou, T., Murray, I., 2017. Masked autoregressive
729 flow for density estimation. *Advances in Neural Information Processing*
730 *Systems* 30, 2335–2344.
- 731 Pedregosa, F., 2016. Hyperparameter optimization with approximate gradi-
732 ent, in: *International Conference on Machine Learning*, pp. 737–746.
- 733 Ren, M., Zeng, W., Yang, B., Urtasun, R., 2018. Learning to reweight ex-
734 amples for robust deep learning, in: *International Conference on Machine*
735 *Learning*, pp. 4334–4343.
- 736 Schölkopf, B., Smola, A., Müller, K.R., 1998. Nonlinear component analysis
737 as a kernel eigenvalue problem. *Neural computation* 10, 1299–1319.
- 738 Shu, J., Meng, D., Xu, Z., 2023. Learning an explicit hyper-parameter predic-
739 tion function conditioned on tasks. *Journal of Machine Learning Research*
740 24, 8818–8891.
- 741 Sober, B., Levin, D., 2020. Manifold approximation by moving least-squares
742 projection (mmls). *Constructive Approximation* 52, 433–478.
- 743 Tenenbaum, J.B., Silva, V.d., Langford, J.C., 2000. A global geometric
744 framework for nonlinear dimensionality reduction. *science* 290, 2319–2323.
- 745 Vincent, P., Larochelle, H., Bengio, Y., Manzagol, P.A., 2008. Extracting and
746 composing robust features with denoising autoencoders, in: *International*
747 *Conference on Machine Learning*, pp. 1096–1103.

748 Wang, W., Carreira-Perpinán, M.A., 2010. Manifold blurring mean shift al-
749 gorithms for manifold denoising, in: 2010 IEEE Computer Society Confer-
750 ence on Computer Vision and Pattern Recognition, IEEE. pp. 1759–1766.

751 Yao, Z., Li, B., Lu, Y., Yau, S.T., 2024a. Single-cell analysis via manifold
752 fitting: A framework for rna clustering and beyond. Proceedings of the
753 National Academy of Sciences 121, e2400002121.

754 Yao, Z., Su, J., Li, B., 2023. Manifold fitting: An invitation to statistics.
755 arXiv preprint arXiv:2304.07680 .

756 Yao, Z., Su, J., Yau, S.T., 2024b. Manifold fitting with cyclegan. Proceedings
757 of the National Academy of Sciences 121, e2311436121.

758 Yao, Z., Xia, Y., 2019. Manifold fitting under unbounded noise. arXiv
759 preprint arXiv:1909.10228 .

760 Yau, S.T., 2008. A survey of calabi-yau manifolds. Surveys in differential
761 geometry 13, 277–318.

762 Zhang, X., Chen, H., Gu, B., Gong, T., Zheng, F., 2024. Fine-grained
763 analysis of stability and generalization for stochastic bilevel optimization,
764 in: Proceedings of the Thirty-Third International Joint Conference on
765 Artificial Intelligence, pp. 5508–5516.

766 Zhao, Q., Shu, J., Yuan, X., Liu, Z., Meng, D., 2023. A probabilistic formu-
767 lation for meta-weight-net. IEEE Transactions on Neural Networks and
768 Learning Systems 34, 1194–1208.

769 Zhen, X., Chakraborty, R., Yang, L., Singh, V., 2021. Flow-based generative
770 models for learning manifold to manifold mappings, in: Proceedings of the
771 AAAI Conference on Artificial Intelligence, pp. 11042–11052.

772 Zhou, X., Pi, R., Zhang, W., Lin, Y., Chen, Z., Zhang, T., 2022. Probabilistic
773 bilevel coresnet selection, in: International Conference on Machine Learning,
774 pp. 27287–27302.

775 Zhu, J.Y., Park, T., Isola, P., Efros, A.A., 2017. Unpaired image-to-image
776 translation using cycle-consistent adversarial networks, in: Proceedings of
777 the IEEE International Conference on Computer Vision, pp. 2223–2232.

Supplementary Materials

Appendix A. Some Notations

Some used notations are summarized in Table A.1.

Table A.1: Main notations

Notations	Descriptions
d_0	the dimension in ambient space
d, d'	the instinct dimension and empirical selected dimension in latent space
m_1, m_2	the sample size of the validation set and training set respectively
$\mathbf{z}_i, \tilde{\mathbf{z}}_i$	the normal and <i>corrupted</i> validation samples respectively
$G_{\mathcal{X}}, G_{\mathcal{Y}}, F_{\mathcal{M}}, D_{\mathcal{X}}, D_{\mathcal{Y}}$	modules of generators, manifold fitting module and discriminators
$\mathbf{m}, \mathbf{s}$	the learned mask and its probability
$\boldsymbol{\theta}_1$	parameters of modules $F_{\mathcal{M}}, D_{\mathcal{X}}, D_{\mathcal{Y}}$
$\boldsymbol{\theta}_2$	parameters of modules $G_{\mathcal{X}}, G_{\mathcal{Y}}$
$\boldsymbol{\theta}$	the whole parameter of the BCGAN model where $\boldsymbol{\theta} = (\boldsymbol{\theta}_2, \boldsymbol{\theta}_1)$
$\mathbb{D}_1, \mathbb{D}_2$	the distributions of validation and training sets respectively
$\mathcal{D}_{meta}$	the validation set with the sizes of m_1
$\mathcal{D}_{train}$	the training set with the sizes of m_2
$A(\mathcal{D}_{meta}, \mathcal{D}_{train})$	stochastic bilevel algorithm with training set $\mathcal{D}_{train}$ and validation set $\mathcal{D}_{meta}$
$R(A(\mathcal{D}_{meta}, \mathcal{D}_{train}))$	the expected population risk at outer-level problem (on validation distribution $\mathbb{D}_1$)
$R_{\mathcal{D}_{meta}}(A(\mathcal{D}_{meta}, \mathcal{D}_{train}))$	the upper empirical risk function (on validation set $\mathcal{D}_{meta}$)
ℓ	the objective function shared for outer-level and inner-level problems
T	the number of total iterations
G_S, G'_S	Update rules for SSGDA algorithm with normal or corrupted sample
η, γ_1, γ_2	the step sizes for updating $\mathbf{s}, \boldsymbol{\theta}_1$ and $\boldsymbol{\theta}_2$, respectively

Appendix B. Proof of Theorem 1

To better illustrate the proof process, we summarize the major steps and lemmas in Figure B.1. Before providing the detailed proof, we first introduce some necessary definitions and lemmas.

Definition 1. (*On-average Argument Stability (Lei and Ying, 2020)*). Let $\mathcal{D}_{meta} = \{\mathbf{z}_1, \dots, \mathbf{z}_{m_1}\}$ and $\tilde{\mathcal{D}}_{meta} = \{\tilde{\mathbf{z}}_1, \dots, \tilde{\mathbf{z}}_{m_1}\}$ be two sets drawn independently from the distribution $\mathbb{D}_1^{m_1}$. For any $i = 1, \dots, m_1$, define $\mathcal{D}_{meta}^{(i)} = \{\mathbf{z}_1, \dots, \mathbf{z}_{i-1}, \tilde{\mathbf{z}}_i, \mathbf{z}_{i+1}, \dots, \mathbf{z}_{m_1}\}$ with at most one different sample w.r.t. $\mathcal{D}_{meta}$. Denote the operator $\mathbb{E}$ as the expectation of $\mathbb{E}_{\mathcal{D}_{meta}, \mathcal{D}_{train}, \tilde{\mathcal{D}}_{train}, A}$. We say a randomized algorithm A is β on-average argument stable if

$$\mathbb{E} \left[\frac{1}{m_1} \sum_{i=1}^{m_1} \left\| A(\mathcal{D}_{meta}, \mathcal{D}_{train}) - A(\mathcal{D}_{meta}^{(i)}, \mathcal{D}_{train}) \right\|_2 \right] \leq \beta.$$

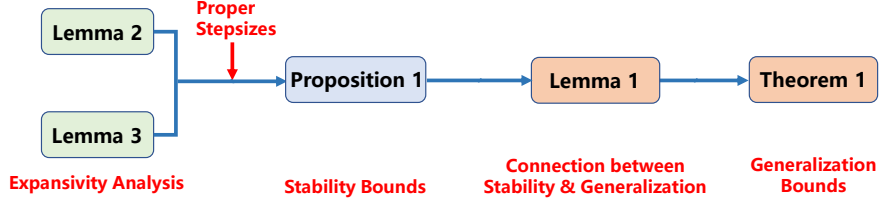


Figure B.1: Proof sketch for Theorem 1.

Remark 7. The on-average argument stability measures the average sensitivity of output parameters of learning models when at most one validation sample is changed. Definition 1 is different from Definition 1 in (Bao et al., 2021), where the uniform stability is evaluated by the drift of prediction error of the hyperparameter optimization algorithm and the boundedness of the loss function is often required.

Lemma 1. Assume algorithm A is β on-average argument stable in expectation and the objective function ℓ is l -Lipschitz continuous w.r.t. $(\mathbf{s}, \boldsymbol{\theta}_1, \boldsymbol{\theta}_2) \in \mathcal{C}_s \times \Theta_1 \times \Theta_2$, and $\mathbf{z} \sim \mathbb{D}_1$. Denote $\mathbb{E}$ as $\mathbb{E}_{A, \mathcal{D}_{meta}, \mathcal{D}_{train}}$. There holds

$$|\mathbb{E}[R(A(\mathcal{D}_{meta}, \mathcal{D}_{train})) - R_{\mathcal{D}_{meta}}(A(\mathcal{D}_{meta}, \mathcal{D}_{train}))]| \leq l\beta.$$

Proof. Algorithm $A(\mathcal{D}_{meta}^{(i)}, \mathcal{D}_{train})$ is independent of $\mathbf{z}_i$. For simplicity, let $\mathbb{E}$ be $\mathbb{E}_{A, \mathbf{z}_i \in \mathcal{D}_{meta}, \mathbf{z} \sim \mathbb{D}_1}$. Since $\mathbf{z}$ and $\mathbf{z}_i$ are drawn from the same distribution and due to the symmetry, we have

$$\begin{aligned}
& \mathbb{E}_A [R(A(\mathcal{D}_{meta}, \mathcal{D}_{train}), \mathbb{D}_1) - R_{\mathcal{D}_{meta}}(A(\mathcal{D}_{meta}, \mathcal{D}_{train}), \mathcal{D}_{meta})] \\
&= \mathbb{E} [\ell(A(\mathcal{D}_{meta}, \mathcal{D}_{train}), \mathbf{z}) - \ell(A(\mathcal{D}_{meta}, \mathcal{D}_{train}), \mathbf{z}_i)] \\
&= \mathbb{E} \left[\ell \left(A \left(\mathcal{D}_{meta}^{(i)}, \mathcal{D}_{train} \right), \mathbf{z}_i \right) - \ell(A(\mathcal{D}_{meta}, \mathcal{D}_{train}), \mathbf{z}_i) \right] \\
&= \mathbb{E} \left[\frac{1}{m_1} \sum_{i=1}^{m_1} \left(\ell \left(A \left(\mathcal{D}_{meta}^{(i)}, \mathcal{D}_{train} \right); \mathbf{z}_i \right) - \ell(A(\mathcal{D}_{meta}, \mathcal{D}_{train}); \mathbf{z}_i) \right) \right] \\
&\leq \mathbb{E} \left[\frac{l}{m_1} \sum_{i=1}^{m_1} \left\| A \left(\mathcal{D}_{meta}^{(i)}, \mathcal{D}_{train} \right) - A(\mathcal{D}_{meta}, \mathcal{D}_{train}) \right\|_2 \right] \leq l\beta,
\end{aligned}$$

where the last inequality follows from the l -Lipschitz assumption and the definition of on-average argument stability. $\square$

805 *B.1 Update Rules*

806 Denote G_S as the update rules for the SSGDA algorithm and set its
 807 maximum iteration number as T . For simplicity, the expectation $\mathbb{E}$ denotes
 808 the expectation on $\mathcal{D}_{meta}$ throughout the proof when not specified.

809 Given step sizes η, γ_1 and γ_2 , the update rules for single-timescale in t -th
 810 iteration can be described as:

$$G_S \left(\begin{bmatrix} \mathbf{s}^t \\ \boldsymbol{\theta}_1^t \\ \boldsymbol{\theta}_2^t \end{bmatrix} \right) := \begin{bmatrix} \mathcal{P}_{\mathcal{C}_s}(\mathbf{s}^t - \eta^t \nabla_{\mathbf{s}} \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t; \mathbf{z}_i)) \\ \boldsymbol{\theta}_1^t - \gamma_1^t \nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t; \mathbf{z}_j) \\ \boldsymbol{\theta}_2^t + \gamma_2^t \nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^{t+1}, \boldsymbol{\theta}_2^t; \mathbf{z}_j) \end{bmatrix}, \quad (\text{B.1})$$

811 where $\mathbf{z}_i \in \mathcal{D}_{meta}$, $\mathbf{z}_j \in \mathcal{D}_{train}$, and $\mathcal{P}_{\mathcal{C}_s}(\cdot)$ is the projection operation from $\mathbf{s}$
 812 to $\mathcal{C}_s = \{\mathbf{s} : 0 \leq \mathbf{s}_i \leq 1, \|\mathbf{s}\|_1 \leq C, i = 1, 2, \dots, p\}$. Thus, we can derive that

$$\|\mathcal{P}_{\mathcal{C}_s}(\mathbf{s})\|_2 \leq \|\mathbf{s}\|_2.$$

813 Denote $\mathbf{s}^{t'}$, $\boldsymbol{\theta}_1^{t'}$ and $\boldsymbol{\theta}_2^{t'}$ as $\mathbf{s}^{(i)}$, $\boldsymbol{\theta}_1^{(i)}$ and $\boldsymbol{\theta}_2^{(i)}$ at t -th iteration, which are
 814 the parameters affected by G_S with corrupted sample $\tilde{\mathbf{z}}$.

815 **Lemma 2.** (*Expansivity when selecting $\mathbf{z}$*) Suppose that Assumptions 1 and 2
 816 hold for the bilevel optimization problem. Consider two sequences of updates
 817 $G_S^1, \dots, G_S^t$ and $(G_S^1)', \dots, (G_S^t)'$ defined in (B.1) with initial points $\mathbf{s}^0 =$
 818 $\mathbf{s}^{0'}$, $\boldsymbol{\theta}_1^0 = \boldsymbol{\theta}_1^{0'}$, $\boldsymbol{\theta}_2^0 = \boldsymbol{\theta}_2^{0'}$. Denote $\eta' = \max\{\eta, \gamma_1, \gamma_2\}$. With step sizes η, γ_1, γ_2 ,
 819 G_S satisfies

$$\left\| G_S \left(\begin{bmatrix} \mathbf{s}^t \\ \boldsymbol{\theta}_1^t \\ \boldsymbol{\theta}_2^t \end{bmatrix} \right) - G_S \left(\begin{bmatrix} \mathbf{s}^{t'} \\ \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right) \right\|_2 \leq (1 + 3L\eta' + 3L^2\eta'^2 + L^3\eta'^3) \left\| \begin{bmatrix} \mathbf{s}^t - \mathbf{s}^{t'} \\ \boldsymbol{\theta}_1^t - \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^t - \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right\|_2.$$

820 *Proof.* Considering the normal scenario with fixed sample, we ignore the
 821 description of $\mathbf{z}_i \in \mathcal{D}_{meta}$ for updating $\mathbf{s}$, and $\mathbf{z}_j \in \mathcal{D}_{train}$ for updating $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2$.

822

According to the L -smoothness properties, we have

$$\begin{aligned}
& \left\| G_S \left(\begin{bmatrix} \mathbf{s}^t \\ \boldsymbol{\theta}_1^t \\ \boldsymbol{\theta}_2^t \end{bmatrix} \right) - G_S \left(\begin{bmatrix} \mathbf{s}^{t'} \\ \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right) \right\|_2 \\
& \leq \left\| \begin{bmatrix} \mathbf{s}^t - \mathbf{s}^{t'} - \eta (\nabla_{\mathbf{s}} \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t) - \nabla_{\mathbf{s}} \ell(\mathbf{s}^{t'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'})) \\ \boldsymbol{\theta}_1^t - \boldsymbol{\theta}_1^{t'} - \gamma_1 (\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t) - \nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'})) \\ \boldsymbol{\theta}_2^t - \boldsymbol{\theta}_2^{t'} + \gamma_2 (\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^{t+1}, \boldsymbol{\theta}_2^t) - \nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1'}, \boldsymbol{\theta}_1^{t+1'}, \boldsymbol{\theta}_2^{t'})) \end{bmatrix} \right\|_2 \\
& \leq \left\| \begin{bmatrix} \mathbf{s}^t - \mathbf{s}^{t'} \\ \boldsymbol{\theta}_1^t - \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^t - \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right\|_2 + \left\| \begin{bmatrix} \eta (\nabla_{\mathbf{s}} \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t) - \nabla_{\mathbf{s}} \ell(\mathbf{s}^{t'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'})) \\ \gamma_1 (\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t) - \nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'})) \\ \gamma_2 (\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^{t+1}, \boldsymbol{\theta}_2^t) - \nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1'}, \boldsymbol{\theta}_1^{t+1'}, \boldsymbol{\theta}_2^{t'})) \end{bmatrix} \right\|_2 \\
& \leq (1 + L\eta) \left\| \begin{bmatrix} \mathbf{s}^t - \mathbf{s}^{t'} \\ \boldsymbol{\theta}_1^t - \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^t - \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right\|_2 + L\gamma_1 \left\| \begin{bmatrix} \mathbf{s}^{t+1} - \mathbf{s}^{t+1'} \\ \boldsymbol{\theta}_1^t - \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^t - \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right\|_2 + L\gamma_2 \left\| \begin{bmatrix} \mathbf{s}^{t+1} - \mathbf{s}^{t+1'} \\ \boldsymbol{\theta}_1^{t+1} - \boldsymbol{\theta}_1^{t+1'} \\ \boldsymbol{\theta}_2^t - \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right\|_2.
\end{aligned}$$

823

Moreover,

$$\begin{aligned}
\left\| \begin{bmatrix} \mathbf{s}^{t+1} - \mathbf{s}^{t+1'} \\ \boldsymbol{\theta}_1^t - \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^t - \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right\|_2 & \leq \left\| \begin{bmatrix} \mathbf{s}^t - \mathbf{s}^{t'} \\ \boldsymbol{\theta}_1^t - \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^t - \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right\|_2 + \eta \left\| \nabla_{\mathbf{s}} \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t) - \nabla_{\mathbf{s}} \ell(\mathbf{s}^{t'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'}) \right\|_2 \\
& \leq (1 + L\eta) \left\| \begin{bmatrix} \mathbf{s}^t - \mathbf{s}^{t'} \\ \boldsymbol{\theta}_1^t - \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^t - \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right\|_2,
\end{aligned}$$

824 and

$$\begin{aligned}
\left\| \begin{bmatrix} \mathbf{s}^{t+1} - \mathbf{s}^{t+1'} \\ \boldsymbol{\theta}_1^{t+1} - \boldsymbol{\theta}_1^{t+1'} \\ \boldsymbol{\theta}_2^t - \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right\|_2 & \leq \left\| \begin{bmatrix} \mathbf{s}^t - \mathbf{s}^{t'} - \eta (\nabla_{\mathbf{s}} \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t) - \nabla_{\mathbf{s}} \ell(\mathbf{s}^{t'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'})) \\ \boldsymbol{\theta}_1^t - \boldsymbol{\theta}_1^{t'} - \gamma_1 (\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t) - \nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'})) \\ \boldsymbol{\theta}_2^t - \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right\|_2 \\
& \leq (1 + L\eta) \left\| \begin{bmatrix} \mathbf{s}^t - \mathbf{s}^{t'} \\ \boldsymbol{\theta}_1^t - \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^t - \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right\|_2 + L\gamma_1 \left\| \begin{bmatrix} \mathbf{s}^{t+1} - \mathbf{s}^{t+1'} \\ \boldsymbol{\theta}_1^t - \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^t - \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right\|_2.
\end{aligned}$$

825

The desired results follow by combining the above inequalities. $\square$

826

Lemma 3. (*Expansivity when selecting $\tilde{\mathbf{z}}$*) Consider two sequences of updates

827

$G_S^1, \dots, G_S^t$ and $(G_S^1)', \dots, (G_S^t)'$ defined in (B.1) with initial points $\mathbf{s}^0 =$

828

$\mathbf{s}^{0'}, \boldsymbol{\theta}_1^0 = \boldsymbol{\theta}_1^{0'}, \boldsymbol{\theta}_2^0 = \boldsymbol{\theta}_2^{0'}$. Define $\delta_t = \sqrt{\|\mathbf{s}^t - \mathbf{s}^{t'}\|_2^2 + \|\boldsymbol{\theta}_1^t - \boldsymbol{\theta}_1^{t'}\|_2^2 + \|\boldsymbol{\theta}_2^t - \boldsymbol{\theta}_2^{t'}\|_2^2}$.

829 When selecting $\tilde{\mathbf{z}}_i$, $G_S^t \neq (G_S^t)'$. Then there holds

$$\begin{aligned} \delta_{t+1} \leq & \min \left\{ \left\| G_S \left(\begin{bmatrix} \mathbf{s}^t \\ \boldsymbol{\theta}_1^t \\ \boldsymbol{\theta}_2^t \end{bmatrix} \right) - G_S \left(\begin{bmatrix} \mathbf{s}^{t'} \\ \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right) \right\|_2, \delta_t \right\} \\ & + \sqrt{\eta^2 \|\nabla_{\mathbf{s}} \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + \gamma_1^2 \|\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + \gamma_2^2 \|\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^{t+1}, \boldsymbol{\theta}_2^t)\|^2} \\ & + \sqrt{\eta^2 \|\nabla_{\mathbf{s}} \ell(\mathbf{s}^{t'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'})\|^2 + \gamma_1^2 \|\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'})\|^2 + \gamma_2^2 \|\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1'}, \boldsymbol{\theta}_1^{t+1'}, \boldsymbol{\theta}_2^{t'})\|^2}. \end{aligned}$$

830 *Proof.* Note that the samples used in G_S and G'_S are not consistent in this
831 case. Observe that

$$\begin{aligned} \delta_{t+1} &= \left\| G_S \left(\begin{bmatrix} \mathbf{s}^t \\ \boldsymbol{\theta}_1^t \\ \boldsymbol{\theta}_2^t \end{bmatrix} \right) - G'_S \left(\begin{bmatrix} \mathbf{s}^{t'} \\ \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right) \right\|_2 \\ &\leq \delta_t + \left\| \begin{bmatrix} \eta (\nabla_{\mathbf{s}} \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t) - \nabla_{\mathbf{s}} \ell(\mathbf{s}^{t'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'})) \\ \gamma_1 (\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t) - \nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'})) \\ \gamma_2 (\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^{t+1}, \boldsymbol{\theta}_2^t) - \nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1'}, \boldsymbol{\theta}_1^{t+1'}, \boldsymbol{\theta}_2^{t'})) \end{bmatrix} \right\|_2 \\ &\leq \delta_t + \left\| \begin{bmatrix} \eta \nabla_{\mathbf{s}} \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t) \\ \gamma_1 \nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t) \\ \gamma_2 \nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^{t+1}, \boldsymbol{\theta}_2^t) \end{bmatrix} \right\|_2 + \left\| \begin{bmatrix} \eta \nabla_{\mathbf{s}} \ell(\mathbf{s}^{t'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'}) \\ \gamma_1 \nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'}) \\ \gamma_2 \nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1'}, \boldsymbol{\theta}_1^{t+1'}, \boldsymbol{\theta}_2^{t'}) \end{bmatrix} \right\|_2 \\ &\leq \delta_t + \sqrt{\eta^2 \|\nabla_{\mathbf{s}} \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + \gamma_1^2 \|\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + \gamma_2^2 \|\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^{t+1}, \boldsymbol{\theta}_2^t)\|^2} \\ &\quad + \sqrt{\eta^2 \|\nabla_{\mathbf{s}} \ell(\mathbf{s}^{t'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'})\|^2 + \gamma_1^2 \|\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'})\|^2 + \gamma_2^2 \|\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1'}, \boldsymbol{\theta}_1^{t+1'}, \boldsymbol{\theta}_2^{t'})\|^2}. \end{aligned}$$

832 Meanwhile,

$$\begin{aligned} &\delta_{t+1} \\ &= \left\| G_S \left(\begin{bmatrix} \mathbf{s}^t \\ \boldsymbol{\theta}_1^t \\ \boldsymbol{\theta}_2^t \end{bmatrix} \right) - G'_S \left(\begin{bmatrix} \mathbf{s}^{t'} \\ \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right) \right\|_2 \\ &\leq \left\| G_S \left(\begin{bmatrix} \mathbf{s}^t \\ \boldsymbol{\theta}_1^t \\ \boldsymbol{\theta}_2^t \end{bmatrix} \right) - G_S \left(\begin{bmatrix} \mathbf{s}^{t'} \\ \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right) \right\|_2 + \left\| \begin{bmatrix} \mathbf{s}^{t'} \\ \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^{t'} \end{bmatrix} - G_S \left(\begin{bmatrix} \mathbf{s}^{t'} \\ \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right) \right\|_2 + \left\| \begin{bmatrix} \mathbf{s}^{t'} \\ \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^{t'} \end{bmatrix} - G'_S \left(\begin{bmatrix} \mathbf{s}^{t'} \\ \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right) \right\|_2 \\ &\leq \left\| G_S \left(\begin{bmatrix} \mathbf{s}^t \\ \boldsymbol{\theta}_1^t \\ \boldsymbol{\theta}_2^t \end{bmatrix} \right) - G_S \left(\begin{bmatrix} \mathbf{s}^{t'} \\ \boldsymbol{\theta}_1^{t'} \\ \boldsymbol{\theta}_2^{t'} \end{bmatrix} \right) \right\|_2 \\ &\quad + \sqrt{\eta^2 \|\nabla_{\mathbf{s}} \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + \gamma_1^2 \|\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + \gamma_2^2 \|\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^{t+1}, \boldsymbol{\theta}_2^t)\|^2} \\ &\quad + \sqrt{\eta^2 \|\nabla_{\mathbf{s}} \ell(\mathbf{s}^{t'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'})\|^2 + \gamma_1^2 \|\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'})\|^2 + \gamma_2^2 \|\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1'}, \boldsymbol{\theta}_1^{t+1'}, \boldsymbol{\theta}_2^{t'})\|^2}. \end{aligned}$$

833 Combining the above results completes the proof. $\square$

834 **Proposition 1.** *Let Assumptions 1, 2 be satisfied and let A be the SS-*
835 *GDA algorithm with T iterations. Denote the non-increasing step sizes by*
836 *$\eta^t = \max\{\eta^t, \gamma_1^t, \gamma_2^t\}$ and $\eta^t \leq c_1/(tL) \leq 1$, where $t \in [1, 2, \dots, T]$ and the*
837 *constant $0 < c_1 \leq 1$. Then algorithm A is β on-average argument-stable in*
838 *expectation with*

$$\begin{aligned} \beta = & \frac{2c_1}{Lm_1} \sqrt{\|\nabla_s \ell(\mathbf{s}^T, \boldsymbol{\theta}_1^T, \boldsymbol{\theta}_2^T)\|^2 + \|\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{T+1}, \boldsymbol{\theta}_1^T, \boldsymbol{\theta}_2^T)\|^2 + \|\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{T+1}, \boldsymbol{\theta}_1^{T+1}, \boldsymbol{\theta}_2^T)\|^2} \\ & + \frac{2c_1}{Lm_1} \sum_{t=0}^T \left(\frac{T}{t+1}\right)^{7c_1} \sqrt{\|\nabla_s \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + \|\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + \|\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^{t+1}, \boldsymbol{\theta}_2^t)\|^2}. \end{aligned}$$

839 *Proof.* Observe that the probability of selecting the normal sample $(\mathbf{z}_i)$ from
840 $\mathcal{D}_{meta}^{(i)}$ reaches $1 - 1/m_1$ and $1/m_1$ for the corrupted sample $(\tilde{\mathbf{z}}_i)$. The update
841 rules (expansivity) of the SSGDA algorithm for different cases are summa-
842 rized in Lemmas 2 and 3. As $L\eta^t \leq c_1/t \leq 1$, we can derive

$$\begin{aligned} & \mathbb{E}[(\delta_{t+1}^{(i)})] \\ \leq & \left(1 - \frac{1}{m_1}\right) (1 + 3L\eta^t + 3L^2(\eta^t)^2 + L^3(\eta^t)^3) \mathbb{E}_A[(\delta_t^{(i)})] \\ & + \frac{1}{m_1} \min\{1 + 3L\eta^t + 3L^2(\eta^t)^2 + L^3(\eta^t)^3, 1\} \mathbb{E}[(\delta_t^{(i)})] \\ & + \frac{1}{m_1} \sqrt{(\eta^t)^2 \|\nabla_s \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + (\gamma_1^t)^2 \|\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + (\gamma_2^t)^2 \|\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^{t+1}, \boldsymbol{\theta}_2^t)\|^2} \\ & + \frac{1}{m_1} \sqrt{(\eta^t)^2 \|\nabla_s \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + (\gamma_1^t)^2 \|\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + (\gamma_2^t)^2 \|\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^{t+1}, \boldsymbol{\theta}_2^t)\|^2} \\ \leq & (1 + 7c_1/t) \mathbb{E}_A[(\delta_t^{(i)})] \\ & + \frac{\eta^t}{m_1} \sqrt{\|\nabla_s \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + \|\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + \|\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^{t+1}, \boldsymbol{\theta}_2^t)\|^2} \\ & + \frac{\eta^t}{m_1} \sqrt{\|\nabla_s \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + \|\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + \|\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^{t+1}, \boldsymbol{\theta}_2^t)\|^2}. \end{aligned}$$

843 Denote $H_t = \mathbb{E}[(\delta_t^{(i)})]$, $A_t = (1 + 7c_1/t)$ and

$$\begin{aligned} B_t = & \frac{\eta^t}{m_1} \sqrt{\|\nabla_s \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + \|\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + \|\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^{t+1}, \boldsymbol{\theta}_2^t)\|^2} \\ & + \frac{\eta^t}{m_1} \sqrt{\|\nabla_s \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + \|\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + \|\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^{t+1}, \boldsymbol{\theta}_2^t)\|^2}. \end{aligned}$$

844 Moreover,

$$\begin{aligned}
H_{T+1} &\leq B_T + B_{T-1} \cdot A_T + B_{T-2} \cdot A_{T-1} \cdot A_T + \cdots + B_0 \cdot \prod_{j=1}^T A_j + H_0 \cdot \prod_{v=0}^T A_v \\
&\leq B_T + \sum_{t=0}^{T-1} B_t \cdot \prod_{j=t+1}^T A_j + H_0 \cdot \prod_{v=0}^T A_v.
\end{aligned}$$

845 Due to the initial settings $\mathbf{s}^0 = \mathbf{s}^{0'}$, $\boldsymbol{\theta}_1 = \boldsymbol{\theta}_1^{0'}$, $\boldsymbol{\theta}_2 = \boldsymbol{\theta}_2^{0'}$, there holds $H_0 =$
846 $\mathbb{E}[(\delta_0^{(i)})] = 0$. It's easy to verify that

$$\prod_{j=t+1}^T \left(1 + \frac{7c_1}{j}\right) \leq \prod_{j=t+1}^T \left\{ \exp\left(\frac{7c_1}{j}\right) \right\} \leq \exp\left(\sum_{j=t+1}^T \frac{7c_1}{j}\right) \leq \exp\left(7c_1 \ln \frac{T}{t+1}\right) = \left(\frac{T}{t+1}\right)^{7c_1}.$$

847 Since $\mathbf{z}_i$ and $\tilde{\mathbf{z}}_i$ follow the same distribution, we know that

$$\begin{aligned}
&\mathbb{E}_{A, \mathcal{D}_{meta}, \mathcal{D}_{train}} \left[\sqrt{\|\nabla_{\mathbf{s}} \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t; \mathbf{z}_i)\|^2 + \|\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t; \mathbf{z}_j)\|^2 + \|\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^{t+1}, \boldsymbol{\theta}_2^t; \mathbf{z}_j)\|^2} \right] \\
&= \mathbb{E}_{A, \mathcal{D}, \mathcal{D}_{meta}, \mathcal{D}_{train}} \left[\sqrt{\|\nabla_{\mathbf{s}} \ell(\mathbf{s}^{t'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'}; \tilde{\mathbf{z}}_i)\|^2 + \|\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1'}, \boldsymbol{\theta}_1^{t'}, \boldsymbol{\theta}_2^{t'}; \mathbf{z}_j)\|^2 + \|\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1'}, \boldsymbol{\theta}_1^{t+1'}, \boldsymbol{\theta}_2^{t'}; \mathbf{z}_j)\|^2} \right].
\end{aligned}$$

848 Combining the above inequalities, we have

$$\mathbb{E}[B_t] \leq \frac{2c_1}{Lm_1} \sqrt{\|\nabla_{\mathbf{s}} \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + \|\nabla_{\boldsymbol{\theta}_1} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|^2 + \|\nabla_{\boldsymbol{\theta}_2} \ell(\mathbf{s}^{t+1}, \boldsymbol{\theta}_1^{t+1}, \boldsymbol{\theta}_2^t)\|^2},$$

849 and $\mathbb{E}[\frac{1}{m_1} \sum_{i=1}^{m_1} \delta_T^{(i)}] \leq \mathbb{E}[B_T] + \sum_{t=0}^{T-1} \left(\frac{T}{t+1}\right)^{7c_1} \mathbb{E}[B_t]$. The desired conclusion
850 holds according to Definition 1. $\square$

851 B.2 Proof of Theorem 1

852 Based on Proposition 1 and the l -lipschitz continuity, we have

$$\beta \leq \frac{2\sqrt{3}l}{7Lm_1} \left(1 + \sum_{t=0}^T \frac{T}{t+1}\right) \leq \mathcal{O}\left(\frac{T \ln(T)}{m_1}\right)$$

853 and

$$|\mathbb{E}[R(A(\mathcal{D}_{meta}, \mathcal{D}_{train})) - R_{\mathcal{D}_{meta}}(A(\mathcal{D}_{meta}, \mathcal{D}_{train}))]| \leq l\beta \leq \mathcal{O}\left(\frac{T \ln(T)}{m_1}\right)$$

854 as $c_1 = 1/7$ and $T > 1$.

When $c_1 \neq 1/7$, there exists an constant $c > 0$ (e.g., the Euler-Mascheroni constant) such that

$$\sum_{t=0}^{T-1} \left(\frac{T}{t+1}\right)^{7c_1} \leq T^{7c_1} \sum_{t=0}^{T-1} \left(\frac{1}{t+1}\right)^{7c_1} \leq T^{7c_1} \left(c + \int_0^{T-1} \left(\frac{1}{t+1}\right)^{7c_1} dt\right) \leq T^{7c_1} \left(c + \frac{T^{1-7c_1}-1}{1-7c_1}\right).$$

Integrating the above estimations with Lemma 1, we get the desired result.

Appendix C. Proof of Theorem 2

Denote the step size $\eta' = \eta_t$. Based on the update rules, there holds

$$\begin{aligned} \|\mathbf{s}^{t+1} - \hat{\mathbf{s}}\|_2^2 &\leq \|\mathbf{s}^t - \eta' \nabla_s \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t) - \hat{\mathbf{s}}\|_2^2 \\ &\leq \|\mathbf{s}^t - \hat{\mathbf{s}}\|_2^2 + (\eta')^2 \|\nabla_s \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)\|_2^2 - 2\eta' \langle \mathbf{s}^t - \hat{\mathbf{s}}, \nabla_s \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t) \rangle. \end{aligned}$$

With the convex property with respect to $\mathbf{s}$, we have

$$\mathbb{E} [\|\mathbf{s}^{t+1} - \hat{\mathbf{s}}\|_2^2] \leq \mathbb{E} [\|\mathbf{s}^t - \hat{\mathbf{s}}\|_2^2] + (\eta')^2 l^2 + 2\eta' \mathbb{E} [\ell(\hat{\mathbf{s}}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t) - \ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)].$$

By rearranging the orders and summing up, we can derive that

$$\begin{aligned} 2\eta' \sum_{t=1}^T \mathbb{E} [\ell(\mathbf{s}^t, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t) - \ell(\hat{\mathbf{s}}, \boldsymbol{\theta}_1^t, \boldsymbol{\theta}_2^t)] &\leq \mathbb{E} [\|\mathbf{s}^0 - \hat{\mathbf{s}}\|_2^2 - \|\mathbf{s}^{T+1} - \hat{\mathbf{s}}\|_2^2] + T(\eta')^2 l^2 \\ &\leq V + T(\eta')^2 l^2. \end{aligned}$$

Without loss of generality, there holds

$$\mathbb{E} [R_{\mathcal{D}_{meta}}(A(\mathcal{D}_{meta}, \mathcal{D}_{train})) - \hat{R}_{\mathcal{D}_{meta}}(A(\mathcal{D}_{meta}, \mathcal{D}_{train}))] \leq \mathcal{O}\left(\frac{V}{\eta' T} + \eta' l^2\right).$$

Recalling Lemma 1 and Theorem 1 with constant $0 < c_1 < 1/7$, the generalization gap could be derived with the order of $\mathcal{O}(T/m_1)$. Moreover, by selecting the number of iterations $T \asymp \sqrt{m_1}$ and the non-increasing step sizes $\eta^{t'} = \max\{\eta^t, \gamma_1^t, \gamma_2^t\} \leq \min\{\frac{c}{\sqrt{T}}, \frac{c_1}{t}\}$ with constant $c > 0$, we can derive that

$$\mathbb{E} [R(A(\mathcal{D}_{meta}, \mathcal{D}_{train})) - R^*(A(\mathcal{D}_{meta}, \mathcal{D}_{train}))] \leq \mathcal{O}(1/\sqrt{m_1}).$$

869 Appendix D. Projected Optimization in Algorithm 2

870 **Inspired by** existing works (Zhao et al., 2023; Zhou et al., 2022), we
871 present the computing procedure associated with Algorithm 2.

872 Given $\mathbf{a} \in \mathbb{R}^d$, in order to project $\mathbf{a}$ to set $\mathcal{C}$, we introduce the following
873 problem

$$\min_{\mathbf{s} \in \mathbb{R}^d} \frac{1}{2} \|\mathbf{s} - \mathbf{a}\|^2, \text{ s.t. } \mathbf{1}^T \mathbf{s} \leq C \text{ and } 0 \leq \mathbf{s}_i \leq 1,$$

874 where $\mathbf{1} = (1, 1, \dots, 1) \in \mathbb{R}^d$ and $\mathbf{s}$ is the ideal output after projection.

875 The above minimization problem can be resolved by the commonly used
876 Lagrangian multiplier method, formulated with

$$\begin{aligned} L(\mathbf{s}, b) &= \frac{1}{2} \|\mathbf{s} - \mathbf{a}\|^2 + b (\mathbf{1}^T \mathbf{s} - C) \\ &= \frac{1}{2} \|\mathbf{s} - (\mathbf{a} - b\mathbf{1})\|^2 + b (\mathbf{1}^T \mathbf{a} - C) - \frac{n}{2} b^2, \end{aligned} \quad (\text{D.1})$$

877 where the auxiliary variable $b \geq 0$ and $0 \leq \mathbf{s}_i \leq 1$.

878 To minimize Eq.(D.1) with respect to $\mathbf{s}$, we consider $\tilde{\mathbf{s}} = \mathbf{1}_{\mathbf{a} - b\mathbf{1} \geq 1} + (\mathbf{a} -$
879 $b\mathbf{1})_{\mathbf{1} > \mathbf{a} - b\mathbf{1} > 0}$. For $b \geq 0$, we introduce two auxiliary functions

$$\begin{aligned} g(b) = L(\tilde{\mathbf{s}}, b) &= \frac{1}{2} \|[\mathbf{a} - b\mathbf{1}]_- + [\mathbf{a} - (b+1)\mathbf{1}]_+\|^2 + b (\mathbf{1}^T \mathbf{a} - C) - \frac{n}{2} b^2 \\ &= \frac{1}{2} \|[\mathbf{a} - b\mathbf{1}]_-\|^2 + \frac{1}{2} \|[\mathbf{a} - (b+1)\mathbf{1}]_+\|^2 + b (\mathbf{1}^T \mathbf{a} - C) - \frac{n}{2} b^2, \end{aligned}$$

880 and

$$\begin{aligned} g'(b) &= \mathbf{1}^T [b\mathbf{1} - \mathbf{a}]_+ + \mathbf{1}^T [(b+1)\mathbf{1} - \mathbf{a}]_- + (\mathbf{1}^T \mathbf{a} - C) - nb \\ &= \mathbf{1}^T \min(1, \max(0, \mathbf{a} - b\mathbf{1})) - C. \end{aligned}$$

881 Based on the monotone decreasing property of $g'(b)$, a bisection method
882 is exploited to solve the equation $g'(b) = 0$ with solution b^* . Because $g(b)$
883 increases in $(-\infty, b^*]$ and decreases in $[b^*, +\infty)$, we can conclude that the
884 maximum of $g(b)$ is obtained at 0 if $b^* \leq 0$ and b^* if $b^* > 0$.

885 Finally, $\mathbf{s}^* = \mathbf{1}_{\mathbf{a} - c^*\mathbf{1} \geq 1} + (\mathbf{a} - c^*\mathbf{1})_{\mathbf{1} > \mathbf{a} - c^*\mathbf{1} > 0} = \min(1, \max(0, \mathbf{a} - c^*\mathbf{1}))$
886 follows by setting $c^* = \max(0, b^*)$.

887 Appendix E. Benchmarks, Baselines and Model Settings

888 E.1 Benchmarks

889 In this paper, 4 synthetic datasets and 2 image datasets are selected for
890 our experiments. Indeed, these datasets have been widely used for validat-

ing the performance of manifold fitting (Yao et al., 2024b, 2023) or sample generation (Papamakarios et al., 2017; Liu et al., 2021).

(1) *Circle, Involute and Torus datasets* are generated by Eq.(12), (13) and (14) respectively, where 400 training samples are generated. The corresponding outputs are generated with each dimension uniformly distributed within (0, 1). The corresponding noisy data is further corrupted by adding noisy dimensions, following a $\mathcal{N}(0, 100)$ distribution.

(2) *Calabi-Yau manifolds* are a class of compact and complex Kähler manifolds. They are highly significant because their geometric representation aligns with the universe model of physicists (Yao et al., 2024b).

(3) *“L” shape images* are constructed using a Gaussian mixture model with component means in an “L” shape. These components are then rotated by 10° . There are 36 images with 32×32 pixels, which are further replicated 15 times, with added white noise.

(4) Frames extracted from *GIFs of Charlie Chaplin and Chow Yun-Fat*. For the former GIF, 34 frames of 160×160 black and white bitmaps are extracted. Those odd-numbered frames with added white noise 20 times per frame, and even-numbered frames remain unchanged as test sets.

In summary, our experimental design has covered a wide range of synthetic and real-world noise scenarios:

- Synthetic Datasets (Circle, Involute, Torus, Calabi-Yau Manifold, “L”-images). To better approximate the real-world noisy scenario, the additive noise, noisy dimensions and the white noise are manually added into the input vector or matrix. Moreover, the corruption of the noisy dimension to the output is also considered.
- Realworld Image Datasets (Charlie Chaplin’s and Chow Yun-Fat’s Frames). The real-world data naturally involves pixel noise and blur in images. Furthermore, we also artificially add white noise into the input images and noisy dimensions into the output embeddings.

E.2 Baselines

The following algorithms are selected as the empirical competitors for manifold fitting tasks.

(1) RealNVP (the real-valued non-volume preserving transformation (Dinh et al., 2017)) introduces a class of invertible functions, real-valued non-volume

preserving transformations, for unsupervised learning of probabilistic models. The algorithm utilizes coupling layers and a multi-scale architecture to model high-dimensional data effectively.

(2) MADE (masked autoencoder for distribution estimation (Germain et al., 2015)) applies the binary mask matrices applied to the weights of the autoencoder, enforcing an autoregressive property where each input dimension is reconstructed based on preceding dimensions in a given order.

(3) MAF (masked autoregressive flow (Papamakarios et al., 2017)) leverages the above MADE as a building block, enabling rapid evaluation and training on parallel architectures.

(4) RoundTrip. Liu et al. (2021) adopts a flexible transformation built by two GANs to map a known distribution onto a manifold.

(5) DAE (denoising autoencoder (Vincent et al., 2008)) introduces a training principle for autoencoders based on the concept of denoising, which learns robust representations of input data invariant to minor, irrelevant changes.

(6) CycleGAN (Cycle-consistent GAN (Zhu et al., 2017)) learns a mapping between two domains without the need for paired training examples, which employs the cycle consistent loss for coherent generalization.

(7) MFCGAN (Cycle-consistent GAN with manifold fitting module (Yao et al., 2024b)) leverages the generative adversarial framework to learn smooth mappings between a low-dimensional latent space and a high-dimensional data space.

(8) MFS (Manifold Fitting based on Statistics (Yao et al., 2023)), utilizes a unique two-step process for estimating the manifold, which involves identifying the contraction direction and evaluating the contracted point based on local averaging techniques.

E.4 Model Settings

The optimization steps for the probabilistic BCGAN have already been summarized in Algorithm 1.

Model Initialization. The lower model architecture defaults to the MFCGAN for initialization. Regarding network topology, the input and output channel dimensions are configured to 1 to accommodate grayscale imaging data. The capacity of the networks is standardized by setting the number of generator filters in the last convolutional layer and discriminator filters in the first layer to 64. Specifically, the generator and primary discriminator utilize a 9-layer Feed-Forward Network, while the secondary discriminator is implemented using the classical and basic PatchGAN model. Furthermore,

network depth is regulated through hyperparameters, with the number of discriminator layers set to 3 and the number of generator layers set to 9.

Module $F_{\mathcal{M}}$. The manifold fitting module $F_{\mathcal{M}}$ involves three (Gaussian) kernels, where the bandwidths are set to 0.05, 0.01, 0.05 as default following (Yao et al., 2024b). Indeed, the detailed optimization process of $F_{\mathcal{M}}$ has been illustrated in the Appendix of (Yao et al., 2024b), which exhibits a theoretical guarantee on its approximation capability (Yao et al., 2023).

Core Size Setting. As for setting C involved in the constraint $\mathcal{C}_s$ where

$$\mathcal{C}_s = \{\mathbf{s} : 0 \leq \mathbf{s}_i \leq 1, \|\mathbf{s}\|_1 \leq C, i = 1, 2, \dots, p\},$$

the ambient dimension D is employed here to initialize $C = D$ as default. For simplicity, we firstly select the optimal value $\hat{C}$ from the searching range of $[D, \lfloor D/2 \rfloor, \lfloor D/4 \rfloor, \dots, 2, 1]$ at the first round, where $\lfloor a \rfloor$ stands for the smallest integer that is not less than a . From our empirical validations, BCGAN is capable of assigning proper masks with a slightly larger C .

Appendix F. Discussions and Limitations

F.1 Discussion on Bilevel Scheme

To address the challenge of noisy dimensions in manifold fitting, we draw inspiration from bilevel optimization frameworks, which have been applied in related domains such as feature selection and robust learning. Existing bilevel methods often focus on tasks like coreset selection (Zhou et al., 2022; Hao et al., 2024; Borsos et al., 2020) or hyperparameter optimization (Pedregosa, 2016; Ji et al., 2021), where the goal is to optimize a primary objective by adjusting a set of hyperparameters (Franceschi et al., 2018). Motivated by these studies, the binary masks are learned via a probabilistic bilevel scheme in this work to remove noisy channels for robust manifold fitting. Theoretical and empirical evaluations validate the competitiveness and robustness of our bilevel manifold fitting scheme.

F.2 Comparisons and Limitations

This section presents a comparative analysis of our BCGAN against numerical analysis-based manifold learning approaches, focusing on robustness, interpretability, computational efficiency, and limitations.

Robustness to Noisy Dimensions. Prior numerical analysis-based methods (Hastie and Stuetzle, 1989; Tenenbaum et al., 2000; Coifman and Lafon,

2006) typically rely on distance metrics computed across the entire ambient space, rendering them sensitive to redundant or noisy features that distort geometric structures (Luo and Hu, 2020). In contrast, our BCGAN employs a bilevel optimization strategy with a probabilistic masking mechanism. This architecture automatically identifies and suppresses irrelevant dimensions, thereby learning robust mappings that are insensitive to such corruptions.

Interpretability. While numerical analysis-based methods offer explicit geometric interpretability through basis functions, deep models are often viewed as “black boxes”. BCGAN mitigates this by providing feature-level transparency through the learned mask, which reveals which latent or ambient dimensions are truly informative. Future work aims to further bridge the gap between deep learning and geometric transparency by integrating gradient-based attribution, interpretable additive models (Gregory et al., 2021), and prototype visualization (Ma et al., 2023).

Parameterization and Intrinsic Dimensionality Assumptions. Unlike traditional approaches that inherently discover the intrinsic dimensionality in an unsupervised manner (Tenenbaum et al., 2000; Papaioannou et al., 2022), neural networks like BCGAN typically require prior knowledge of parameterization and hyperparameters. To alleviate this limitation, our method incorporates a sparsity-inducing penalty on the masks, allowing practitioners to set the target dimension slightly larger than the estimated actual value.

Noise Distribution Assumptions and Generalization. Theoretically, the manifold fitting module in BCGAN assumes Gaussian noise to provide strict guarantees in statistics (Yao et al., 2023, 2024b). However, real-world noise is often complex and non-Gaussian (Luo and Hu, 2020), and CycleGAN-like techniques may struggle with generalization on sparse data. Despite the theoretical assumption, our experiments demonstrate that BCGAN maintains strong robustness across various noise distributions. We attribute this to the manifold fitting module and adversarial nature of the training process, which prioritizes learning the underlying geometric structure over fitting specific noise parameters, ensuring effective generalization in sparse scenarios.

In summary, the BCGAN framework effectively addresses the limitations of traditional methods in handling noise and computational load, while offering a unique perspective on model interpretability. These attributes collectively establish a robust and efficient pathway for manifold fitting.